PROJECT REPORT

FraudLens: AI-Based Credit Card Fraud Detection System

An explainable hybrid machine-learning system that detects suspicious card transactions in real time, scores their risk and explains every decision

Hybrid ML (XGBoost + Random Forest + Isolation Forest)

Explainable AI (SHAP)

Real-time REST API

React dashboard



DOMAIN
Cyber Security / Forensics · Machine Learning / Data Security

TEAM
[Team member names and roll numbers]

GUIDE
[Guide name]

INSTITUTION
[College / University, Department]

DATE
September 2026

REPOSITORY
github.com/Prateek22672/AI-Based-Credit-Card-Fraud-Detection-System

Contents

- Executive summary
1 Introduction
2 Literature review
3 Choosing the solution: web, app or model?
4 System architecture
5 Datasets and exploratory analysis
6 Methodology
7 Experiments and results
8 How FraudLens detects scams
9 What we do better, and how we prove it
10 Implementation
11 How to run, test, present and deploy
12 Limitations and threats to validity
13 Future scope
14 Conclusion
- References
- Appendix A: Configuration and hyper-parameters
- Appendix B: Work distribution

Executive summary

Card fraud is rare (about 1 transaction in 170 in our data), changes over time, and is expensive: global card fraud losses reached **USD 33.41 billion in 2024** (Nilson Report, Jan 2026). A useful detector must catch most fraud, raise few false alarms, decide within the card-authorisation budget (well under a second) and explain itself to analysts and customers.

FraudLens is a complete system, not only a model. Every card transaction is turned into behavioural features computed from that card's own history. Three detectors score it: XGBoost and Random Forest learn known fraud patterns, and Isolation Forest flags behaviour unlike normal spending. A stacking meta-learner combines them into a calibrated fraud probability, which becomes a 0-100 risk score and a decision (**Approve / Step-up verification / Block**). A safety layer of documented policy rules and a novelty alarm can raise that decision for attack types that never appear in the training data. Every decision comes with plain-language reasons from SHAP and is stored in an auditable log. A FastAPI service scores transactions in real time, and a React dashboard lets analysts and reviewers watch, test and investigate.

0.979	92.5%	94.6%	99.6%	256.6 ms
PR-AUC (test) unseen Apr-Jun 2020 data	**Frauds caught** at the validation-tuned threshold	**Alert precision** alerts that are real fraud	**Fraud value stopped** at the cost-optimal threshold	**End-to-end latency** per transaction, incl. SHAP, 1 thread

Verdict: web, app or model?

Build a web platform (real-time REST API + web dashboard) powered by a hybrid ML model. Fraud scoring has to run on the bank's server, inside the payment-authorisation path, with access to every card's history. That rules out a phone app as the core: an app would expose the model to reverse engineering and cannot see other transactions. A single "special" deep model is not the answer either: on the same data and split, our deep models (MLP PR-AUC 0.960, 1D-CNN 0.933) were less accurate, slower and harder to explain than gradient-boosted trees (XGBoost 0.978). A mobile app remains a good future *client* of the same API, for customer "Was this you?" alerts.

What is new compared with the reviewed work

- **Hybrid of known-pattern and unknown-pattern detection** (supervised + anomaly), combined by a learned stacking model instead of fixed voting.
- **Defence in depth, measured**: we found that stacking gives the anomaly detector almost no weight, so a novelty alarm and documented policy rules (velocity burst, impossible travel, card testing) now escalate attacks absent from the training data, at a measured cost of 0.81 extra step-ups per day on the test period.
- **Calibrated risk score with money-based thresholds** instead of a bare fraud/not-fraud label; three action tiers mapped onto the team's 0-30 / 31-60 / 61-100 bands.
- **Behavioural feature store** used identically in training and in the live API (tested for exact parity), so there is no training/serving skew.
- **Honest evaluation**: time-based split with label-delay gaps, thresholds chosen on validation data only, PR-AUC/MCC/money metrics, bootstrap confidence intervals and paired significance tests.
- **Proof of how common shortcuts inflate results**: the same model scores near-perfect only when SMOTE is applied before a random split (Section 7.3).
- **Explainability built into the product**: every decision carries human-readable reasons, mitigating factors and a forensic audit record.
- **Responsible and secure by design**: gender and age are excluded from decisions, card numbers are tokenised, and the API validates input and can require a key.

1 Introduction

1.1 Background and motivation

Digital payments have made card-not-present (online) fraud the dominant form of card fraud. In Europe, 80% of the value of card fraud on SEPA cards in 2019 came from card-not-present transactions (ECB, 2021), and total reported payment fraud in the EEA reached EUR 4.2 billion in 2024 (EBA-ECB, 2025). In the UK, remote-purchase fraud cases grew 22% in 2024 to nearly 2.6 million (UK Finance, 2025). In India, card/internet/digital-payment frauds were the most numerous category of bank fraud reported to the Reserve Bank of India in 2023-24 (28,836 cases, 80.4% of all cases) and again in 2024-25 (RBI Annual Report 2025-26).

Regulation turns this into a direct cost for banks. Under the RBI's 2017 circular on limiting customer liability in unauthorised electronic banking transactions, the customer bears zero liability when the fault lies with the bank or with a third-party breach reported within three working days; the burden of proof is on the bank, and banks must operate a "robust and dynamic fraud detection and prevention mechanism". The RBI Master Direction on Digital Payment Security Controls (2021) requires documented rules and parameterised alerts on transaction velocity, high-risk merchant categories, time zones, geo-location and behaviour (paras 36-37), and allows adaptive, risk-based authentication (para 34). FraudLens computes exactly these signals, learns how to weigh them from data, and its STEP-UP tier is that risk-based authentication.

1.2 Problem statement

Build a system that, for every card transaction and within the authorisation budget, (a) estimates the probability that it is fraudulent, (b) turns it into an actionable decision that balances missed fraud against customer friction, and (c) explains the decision, all while coping with extreme class imbalance, fraud patterns that change over time and new fraud types never seen in training.

1.3 Objectives

1. Engineer behavioural features that capture how each cardholder normally behaves, computed only from past data.
2. Compare classical ML, ensembles, deep learning and anomaly detection fairly on the same chronological split.
3. Compare imbalance-handling strategies (none, class weighting, undersampling, SMOTE) and choose by evidence.
4. Combine supervised and anomaly detectors into a calibrated hybrid risk engine with cost-based decision tiers.
5. Explain every prediction with SHAP and convert explanations into readable reason codes.
6. Deliver a real-time REST API, an auditable decision log and a web dashboard for monitoring, testing and review.
7. Prove each claim with experiments, statistical tests and a comparison against published results.

1.4 Scope and assumptions

- Real bank data cannot be shared, so we use two public datasets: the Sparkov simulated dataset (named, behavioural fields) as the primary dataset, and the ULB/Worldline European dataset (real but anonymised with PCA) as the benchmark most papers use.
- Amounts are in USD because Sparkov is US-based; the method is currency-independent.
- Cost model: a missed fraud costs its amount; every alert costs $5 (review / OTP / friction). A sensitivity analysis covers $1-$25.

- Labels are assumed to be confirmed about 7 days after a transaction, which is modelled by gaps between the train, validation and test periods.

2 Literature review

We studied the team's 20 selected papers and 44 further verified sources. The separate **Literature Mapping Report** traces every paper to what we adopted and where it lives in the code; this chapter summarises the findings that shaped the design.



Figure 1. Evolution of fraud-detection technology in the reviewed literature (T# = team paper number)

Table 1. Research gaps and how FraudLens addresses them

Gap found in the literature	Evidence	FraudLens response
Accuracy reported on 99.8%-genuine data	T1 (99.87% acc.), T3 (99.96% acc. but 83% recall), T7	PR-AUC, recall, precision, F1, MCC and money saved
Oversampling before the split / random splits	Santos 2018; Hayat & Magnier 2025	Chronological split, resampling only inside training (E3 shows the effect)
Single model, fixed voting	T3, T4, T7, T8	Learned stacking of supervised + anomaly detectors (E4)
Only known fraud patterns	T11, Carcillo 2021	Isolation Forest novelty layer tested on a held-out fraud type (E5)
Black-box decisions	T13, T16, Psychoula 2021	TreeSHAP reason codes in every API response
Offline notebooks only	T16, T20	Real-time API, online feature store, dashboard, audit log
Heavy DL for tabular data	T1, T2, T9; Grinsztajn 2022	DL tested as a baseline; trees win on accuracy, speed and explainability (E1)
Probabilities distorted by resampling	van den Goorbergh 2022; Dal Pozzolo 2015	Calibrated stacking on the natural class balance + cost thresholds (E2, E7)

2.1 Research questions

Table 2. Research questions and the experiments that answer them

#	Question	Answered by
RQ1	Which model family detects fraud best on tabular transaction data, and at what speed?	E1, E6
RQ2	Is SMOTE (used in most reviewed papers) better than class weighting?	E2
RQ3	How much do common evaluation shortcuts inflate reported results?	E3
RQ4	Do behavioural features and the hybrid combination add real value?	E4
RQ5	Can the system catch a fraud type it was never trained on?	E5
RQ6	What is the business value, and is performance stable over time?	E7

3 Choosing the solution: web, app or model?

The team asked whether the project should be a website, a mobile app or a special model. We scored the options against what a fraud-detection system must do in practice (5 = best).

Table 3. Decision matrix

Criterion	Mobile app only	Stand-alone model / notebook	Deep-learning "special" model	Web platform + API + hybrid ML (chosen)
Runs inside the bank's payment authorisation path	1	1	3	5
Sees every card's history (velocity, spend patterns)	1	3	4	5
Model protected from reverse engineering / evasion	1	4	4	5
Sub-second latency on ordinary servers	3	3	2	5
Explains each decision to analysts and customers	2	2	1	5
Serves analysts, reviewers and bank systems	2	1	2	5
Easy for the team to deploy on its own domain	2	3	2	4
Future extension (mobile alerts, streaming, retraining)	3	1	3	5
Total (out of 40)	**15**	**18**	**21**	**39**

Verdict

Fraud detection is a server-side, real-time decision service. The right product is an **API-first web platform**: the API is what payment systems call, and the dashboard is where people monitor, investigate and present. The "special" part is not a new neural network but the **hybrid risk engine**: behavioural features, three complementary detectors, a calibrated risk score with cost-based tiers, and built-in explanations.

Independent evidence agrees: tree ensembles remain state of the art on tabular data (Grinsztajn et al., 2022; Shwartz-Ziv & Armon, 2022). In our own benchmark (E1) XGBoost reached PR-AUC 0.978 in 2.0 minutes of training, versus 0.960 for the MLP (11 min) and 0.933 for the 1D-CNN (58 min for at most 8 epochs; it trained while other experiments shared the CPU, so its time is an upper bound).

4 System architecture



Figure 2. How one transaction flows through FraudLens

Table 4. Components and technology stack

Layer	Technology	Responsibility
Feature store	Python (fraudlens/features.py)	Per-card rolling state; the same code builds training data and live features
Detectors	XGBoost 3.2, scikit-learn 1.9 (RF, IF)	Known-pattern classification + novelty detection
Risk engine	scikit-learn logistic stacking	Calibrated probability, risk score, decision tiers
Safety layer	Python (fraudlens/policy.py)	Documented policy rules P1-P3 + novelty alarm N1; can only raise a decision
Explainability	XGBoost TreeSHAP (pred_contribs), SHAP library for plots	Per-decision reasons; global importance
API	FastAPI + Uvicorn, Pydantic validation	Real-time scoring, stream replay, scenarios, case review
Decision log	SQLAlchemy 2 on SQLite (PostgreSQL via one env var)	Forensic audit trail + analyst feedback
Dashboard	React 18 + Vite + Tailwind CSS + Recharts	Live monitor, tester, scenarios, cases, results, report
Research	pandas, imbalanced-learn, PyTorch (baselines), matplotlib	Experiments E1-E7, figures, reports
Deployment	Docker (single container)	API + built dashboard on any VPS or cloud host

4.1 Life of a transaction

1. The payment system (or the dashboard) sends `POST /api/score` with the card token, amount, category, merchant, location and time.
2. The feature store reads the card's history (last-7-day window, running mean and variance, category and merchant counts, last location) and computes the features in about 0.07 ms.
3. XGBoost and Random Forest return fraud probabilities; Isolation Forest returns an anomaly score, expressed as a percentile of normal behaviour.
4. The meta-learner turns the three scores into one calibrated probability, mapped to a 0-100 risk score and a tier.
5. The safety layer checks the documented policy rules (velocity burst, impossible travel, card testing) and the novelty alarm; any hit raises the decision to at least Step-up.
6. TreeSHAP attributes the XGBoost score to features, and related features are grouped into sentences. Alerts and manual scores are explained immediately; approved stream transactions are explained on demand when opened, because exact TreeSHAP over the full tree ensemble costs more than the models themselves.

7. The decision, feature vector, reasons and model version are written to the decision log; alerts enter the case queue.
8. If committed, the transaction is appended to the card's history so the next transaction sees it.

4.2 Security and forensics by design

Table 5. Security and forensic controls

Control	Implementation
Card-data minimisation	Card numbers are replaced by a keyed HMAC-SHA256 token; only the last 4 digits are shown. Names, street, job and zip are dropped at load time.
Forensic audit trail	Every decision stores the timestamp, model version, full feature vector, probability, detector scores, reasons and reviewer verdict, so any decision can be reproduced and justified in a dispute.
Access control	Optional API key (X-API-Key) on all state-changing endpoints; CORS restricted by configuration.
Input validation	Pydantic schemas bound every field (amount range, category whitelist, coordinates).
Fairness	Gender and age are never used for decisions (the cost in accuracy is measured in E4).
Sandboxing	What-if scoring only reads the card's history; scam scenarios run on a sandbox copy of the card's state, never on live data.

5 Datasets and exploratory analysis

Table 6. Datasets

Property	Sparkov (primary)	ULB European (benchmark)
Transactions	1,296,675	284,807
Frauds	7,506 (0.58%)	492 (0.173%)
Period	2019-01-01 to 2020-06-21	2 days (September 2013)
Cards / merchants	983 cards, 693 merchants	not provided
Features	Named: amount, category, merchant, time, customer and merchant location	PCA components V1-V28 + time + amount
Why we use it	Behavioural features and readable explanations are possible	Standard benchmark in most papers (T3, T10, Shanaa 2025)

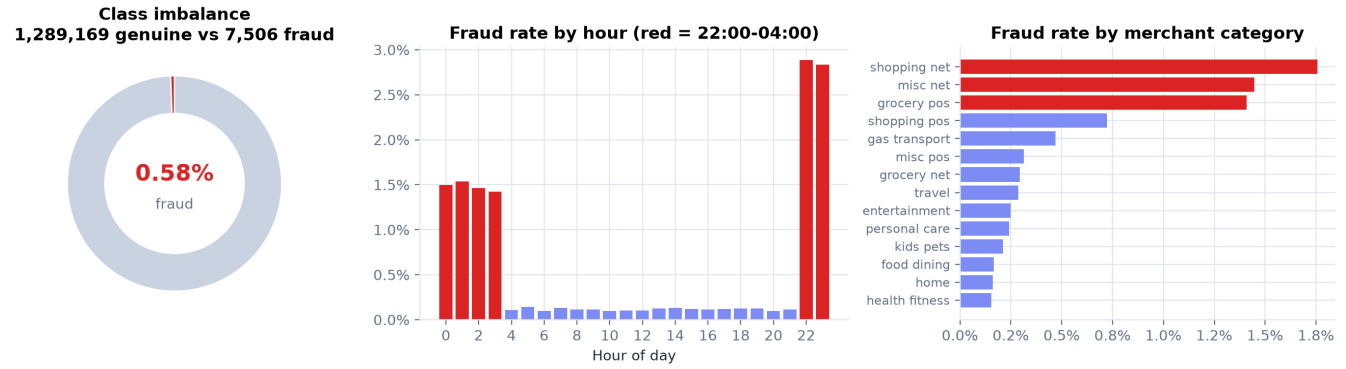


Figure 3. Sparkov: extreme imbalance, fraud concentrated between 22:00 and 04:00, and in online shopping, grocery (in-store) and online miscellaneous categories.

- **Night-time**: the fraud rate is 1.94% between 22:00 and 04:00 versus 0.11% during the day.
- **Amount**: frauds average $531 against $68 for genuine transactions.
- **Bursts**: 762 cards were compromised, each with about 10 fraudulent transactions within roughly two days. Velocity features are therefore essential.
- **Category**: online shopping, in-store grocery and online miscellaneous have the highest fraud rates.

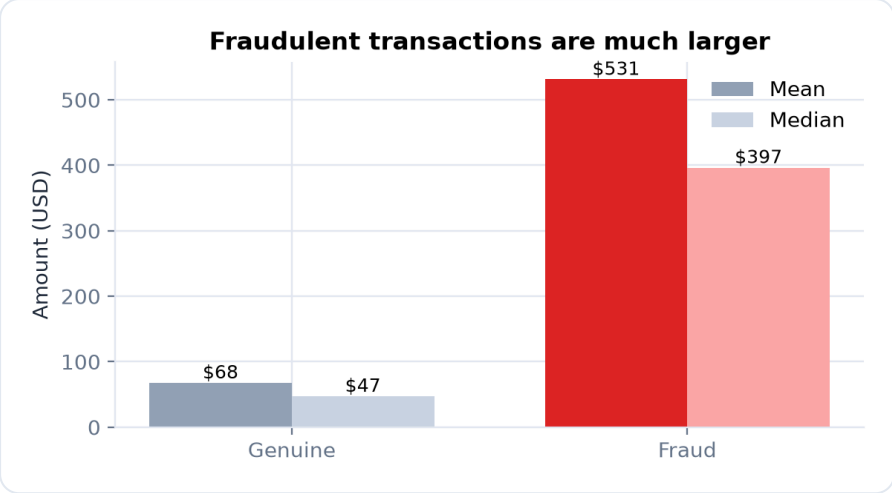


Figure 4. Fraudulent transactions are several times larger than genuine ones.

5.1 Chronological split with label-delay gaps

Table 7. Time-based split (Sparkov)

Block	From	To	Transactions	Frauds	Used for
Train	2019-01-01	2019-12-31	924,850	5,220	Fitting the detectors
Gap	2020-01-01	2020-04-07	28,241	139	7-day gaps: history only, never scored
Val	2020-01-08	2020-03-31	160,810	1,055	Early stopping, meta-learner, thresholds
Test	2020-04-08	2020-06-21	182,774	1,092	Final evaluation + live demo stream

Why a chronological split

A random split lets the model learn from the future and puts transactions from the same fraud burst in both training and test data, which makes results look better than they are. We train on the past, validate on the next quarter and test on the most recent quarter, with 7-day gaps between them because fraud labels take days to be confirmed (Dal Pozzolo et al., 2018; Le Borgne et al., 2022). The test period is also what the dashboard replays live, so the demo runs on data the model has never seen.

6 Methodology

6.1 Behavioural feature engineering

We compute 27 numeric features plus 14 category indicators; the deployed model uses 39 of them (age and gender excluded). Each is computed from the card's past only (Whitrow et al., 2009; Bahnsen et al., 2016).

Table 8. Behavioural and transaction features

Feature(s)	Meaning	Fraud signal it captures
amt, log_amt	Transaction amount (raw and log)	Fraudsters maximise value before the card is blocked
hour, is_night, day_of_week, is_weekend	When the transaction happens	Fraud concentrates late at night when owners sleep
is_online, cat_* (14)	Merchant category and channel (card-not-present)	Stolen card details are mostly used online
log_city_pop, dist_home_km	Home-city size; merchant distance from the cardholder's home	Use far from the usual area
card_history_len	How many past transactions the card has	New cards carry less behavioural evidence
log_secs_since_last	Time since the card's previous transaction	Rapid-fire use (bursts) or a dormant card waking up
cnt_1h, cnt_24h, cnt_7d	Velocity: transactions in the last 1 h / 24 h / 7 days	Card-testing bursts and spending sprees
sum_24h, cnt_24h_vs_usual	Money spent in 24 h; activity vs the card's usual daily rate	Account takeover drains the card quickly
amt_to_card_mean, amt_zscore	Amount relative to this card's own average and spread	Unusual for *this* customer, not just large
amt_to_cat_mean	Amount vs this card's usual spend in the same category	A $900 grocery bill for a $60 grocery shopper
cat_share, is_new_category	How often the card uses this category; first use	Fraudsters buy what they can resell, not what the owner buys
is_new_merchant	First purchase at this merchant	New merchants are riskier than regular ones
night_share	The card's normal share of night-time use	Night use matters only for customers who never shop at night
dist_prev_km, speed_kmh	Distance and implied speed since the previous transaction	Impossible travel = cloned card

Fast and consistent

The feature store processed all 1,296,675 transactions in 87 s (67 microseconds per transaction). A unit test replays the test stream through the API's feature-store snapshot and checks the features match the training matrix exactly.

6.2 Class-imbalance strategy

Fraud is 0.58% of Sparkov transactions and 0.17% of ULB. Most reviewed papers oversample with SMOTE. We compared four strategies (E2) and deploy **class weighting** in the base learners followed by **calibration on the natural class balance** in the meta-learner. This keeps probabilities honest, which the risk score depends on (van den Goorbergh et al., 2022; Elkan, 2001).

6.3 Models compared

Table 9. Models

Model	Type	Role in the study
Legacy rules	4 expert if-then rules (amount, night-online, velocity, spend ratio)	The "existing system" baseline
Logistic Regression	Linear	Classical baseline (T4, T8)
Decision Tree	Single tree	Interpretable baseline (T8)
Random Forest	Bagged trees	Second opinion in the hybrid (T5, T18)
XGBoost	Gradient-boosted trees	Main classifier (T3, T7)
LightGBM	Gradient-boosted trees	Alternative booster
Isolation Forest	Unsupervised anomaly detection	Novelty layer in the hybrid
Deep MLP	3-layer neural network (PyTorch)	Deep-learning baseline (T10, T12)
1D-CNN	Convolutional network over the feature vector	Reproduces the idea of T1
FraudLens Hybrid	Stacked XGB + RF + IF	Proposed system

6.4 Hybrid risk engine

The three detectors are trained on the training period (Isolation Forest on genuine transactions only). On the validation period, a logistic-regression meta-learner is fitted on `[logit(p_xgb), logit(p_rf), anomaly_z]`, so the weights come from data rather than guesswork (Wolpert, 1992). Because the validation period has the natural fraud rate, the output is a calibrated probability. Two thresholds are chosen on out-of-fold validation predictions:

- **Alert (step-up) threshold** = 0.0142: minimises missed-fraud dollars + $5 x alerts.
- **Block threshold** = 0.2065: the lowest probability at which alerts are at least 90% precise.

Table 10. Risk tiers: the team's original bands, now anchored to learned thresholds

Risk score	Level	Action	Probability range
0-30	LOW	Approve	< 0.0142
31-60	MEDIUM	Step-up verification (OTP / 3-D Secure) or analyst review	0.0142 - 0.2065
61-100	HIGH	Block and open a case	>= 0.2065

Learned stacking weights: XGBoost 0.90, Random Forest 0.18, Isolation Forest -0.09 (intercept -0.75), on logit / z-score scales.

6.5 Safety layer: defence in depth

The stacking weights above revealed a design flaw that most hybrid papers do not report. The meta-learner is fitted on *labelled* fraud, which XGBoost already explains, so it gives the Isolation Forest almost no weight (-0.09). Stacking alone therefore cannot use the anomaly detector for its real purpose: fraud types that never appeared in the labels. In the scam replays (Section 8.1), the anomaly detector rated a card-testing cash-out as more unusual than 99.9% of normal spending, yet the stacked score stayed below the alert line.

FraudLens therefore adds a safety layer that can only *raise* a decision to at least Step-up, never approve anything: a novelty alarm with its own route, plus three documented, parameterised policy rules of the kind the RBI Master Direction (2021, paras 36-37) asks banks to run.

Table 11. Safety-layer rules (fraudlens/policy.py); every hit raises the decision to at least STEP-UP

Rule	Fires when	Attack it targets	Basis
P1 Burst then jump	3+ transactions in the past hour and an amount 5x or more the card's average	Account takeover / cash-out after card testing	Velocity rules (RBI MD para 36-37); Bahnsen 2016
P2 Impossible travel	500 km+ from the previous transaction at an implied speed above 800 km/h	Cloned (skimmed) card	Geo-location rules (RBI MD para 36-37)
P3 Micro-charge burst	A tiny (< $5) online charge after 3+ others in the past hour	Card testing (enumeration)	Industry card-testing pattern
N1 Novelty alarm	Anomaly percentile at least 99.95% of genuine training transactions	New, unlabelled fraud patterns	Liu 2008; Carcillo 2021

Cost on the unseen test period: the layer adds 0.81 step-ups per day beyond the ML's own alerts (Section 7.8). The rule parameters come from domain reasoning and were checked on the training data; the 99.95th-percentile novelty threshold is a business setting chosen for a low alert cost, not tuned for detection.

6.6 Explainable AI

For each decision, TreeSHAP (Lundberg et al., 2020) attributes the XGBoost log-odds to every feature exactly, using the same trees the model predicts with, so the contributions sum to the model output (a unit test checks this). Related features are grouped (for example all velocity counters), and each group with a meaningful push becomes a sentence built from the transaction's real values, for example "Amount $1,150.00 is 19.3x this card's average spend ($60)" or "4 other transactions on this card in the past hour". Groups pushing towards genuine become "looks genuine because" statements.

6.7 Evaluation protocol

Table 12. Metrics

Metric	Meaning	Why
PR-AUC (average precision)	Area under the precision-recall curve	Primary metric for rare events (Saito & Rehmsmeier, 2015)
Recall	Share of frauds caught	Missed fraud = direct loss
Precision	Share of alerts that are fraud	False alarms annoy customers and cost review time
F1 / MCC	Balance of precision and recall / all four confusion cells	MCC is robust to imbalance (Chicco & Jurman, 2020)

Metric	Meaning	Why
ROC-AUC	Ranking quality over all thresholds	Reported, but optimistic on imbalanced data
Money saved	Fraud $ stopped minus alert costs	What the bank actually cares about (Bahnsen 2016; Hoppner 2022)
Latency	Milliseconds per transaction	Authorisation happens in real time

- Thresholds are always chosen on validation data and applied unchanged to the test period.
- 95% confidence intervals come from 500 bootstrap resamples of the test set; model differences use a paired bootstrap.
- All models share the same features, split and seed.

7 Experiments and results

7.1 E1: model benchmark

Table 13. E1: test-period results on Sparkov (threshold tuned on validation for F1)

Model	PR-AUC [95% CI]	ROC-AUC	Precision	Recall	F1	MCC	Fit (s)	Latency p50 (ms)
FraudLens Hybrid (XGB + RF + IF)	**0.979 [0.973-0.984]**	**0.999**	**0.946**	**0.925**	**0.935**	**0.935**	**270**	**209.39**
XGBoost	0.978 [0.973-0.984]	0.999	0.937	0.937	0.937	0.936	123	1.67
LightGBM	0.980 [0.975-0.985]	1.000	0.948	0.942	0.945	0.945	114	2.03
Random Forest	0.943 [0.933-0.952]	0.998	0.896	0.851	0.873	0.872	121	38.76
Deep MLP (PyTorch)	0.960 [0.952-0.968]	0.999	0.914	0.890	0.902	0.902	633	-
1D-CNN (PyTorch)	0.933 [0.922-0.944]	0.998	0.892	0.827	0.858	0.858	3456	-
Decision Tree	0.787 [0.760-0.809]	0.943	0.895	0.743	0.812	0.814	29	0.43
Logistic Regression	0.441 [0.415-0.473]	0.990	0.481	0.735	0.581	0.592	3	1.29
Isolation Forest (unsupervised)	0.421	0.965	0.478	0.514	0.495	0.492	20	65.36
Rule-based (legacy bank rules)	0.248	0.795	0.648	0.268	0.380	0.415	0	-

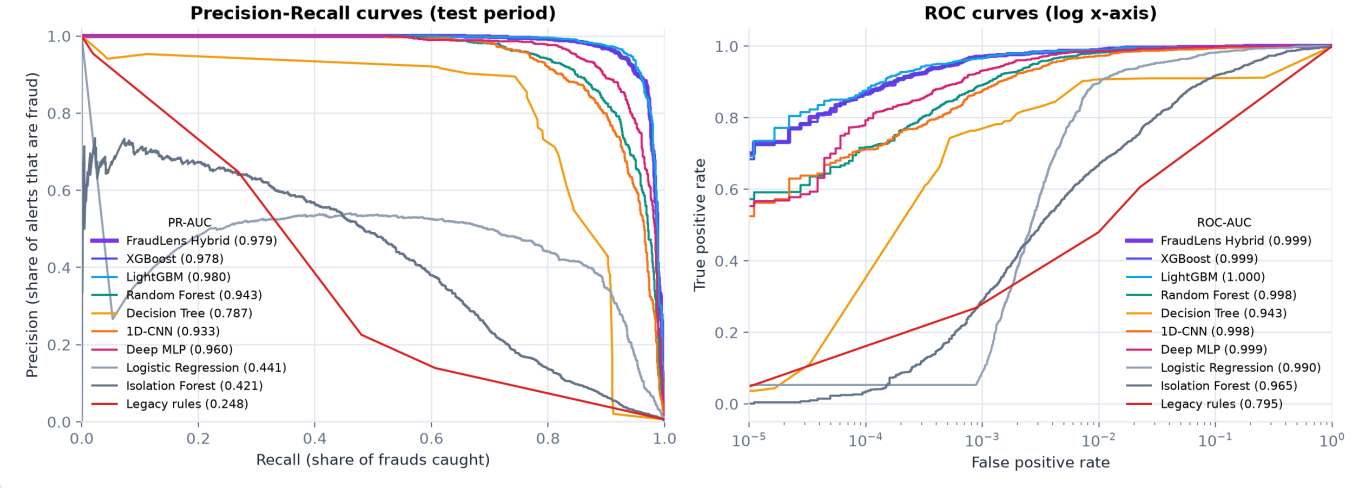


Figure 5. Precision-recall and ROC curves on the unseen test period.

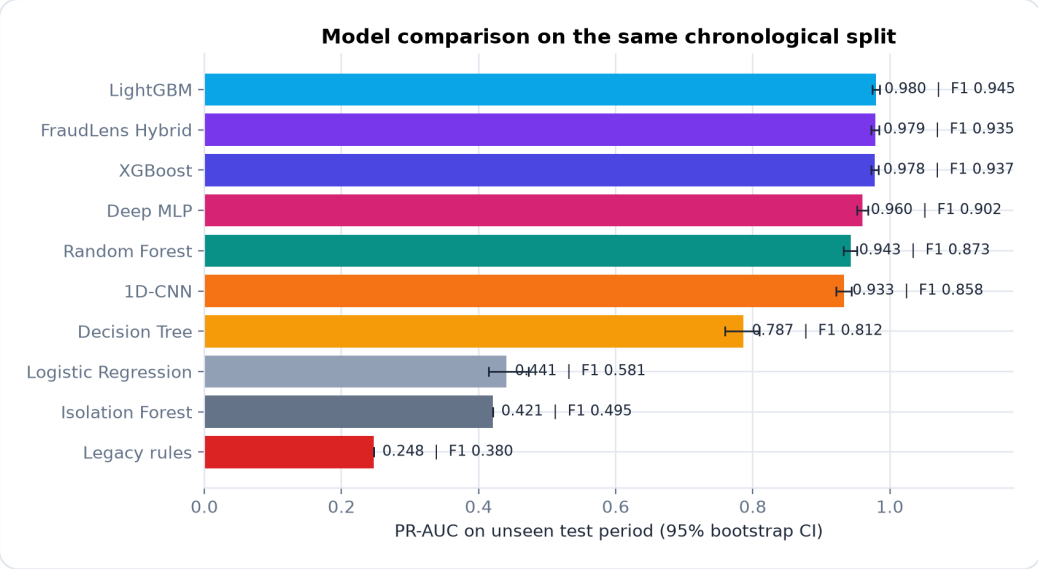


Figure 6. PR-AUC with 95% bootstrap confidence intervals.

Finding (RQ1)

Gradient-boosted trees lead: XGBoost PR-AUC 0.978 and the hybrid 0.979, against 0.441 for Logistic Regression and 0.248 for the legacy rules. The hybrid catches 92.5% of frauds with 94.6% precision. The hybrid and XGBoost alone are statistically indistinguishable on PR-AUC (difference +0.0001, 95% CI [-0.0002, 0.0005]). The hybrid's value is therefore in calibrated probabilities, decision tiers, a second independent opinion and, through the safety layer, coverage of fraud absent from the training data (E5, Section 7.8), not in raw ranking. LightGBM (PR-AUC 0.980) matches XGBoost within the confidence interval; XGBoost stays the main classifier because its exact TreeSHAP is built into the predictor, which keeps the per-decision explanations in the API cheap. LightGBM was numerically unstable with a 177x class weight until the leaf step was capped (Appendix A), a practical finding in itself. The deep models scored lower and took far longer to train: 11 min (MLP) and 58 min (1D-CNN, at most 8 epochs, with other jobs sharing the CPU) against 2.0 min for XGBoost.

Latency was re-measured separately on an idle laptop CPU, one thread, one transaction per call. Scoring by the hybrid's three models and meta-learner takes 209.4 ms (median; p95 424.8 ms). The full API path, including the feature store, the safety layer, the SHAP explanation and the database write, takes 256.6 ms (p95 526.7 ms), and the replay stream sustains about 730 transactions per second. XGBoost itself answers in about 2 ms; the single-call cost of the hybrid comes from scikit-learn's Random Forest and Isolation Forest, which loop over their trees in Python; in batches the whole path costs about 1.4 ms per transaction. That is comfortably sub-second, which is enough for this platform, but production issuers usually aim much lower, so compiling the two forests with a tree runtime such as treelite or ONNX Runtime is the listed optimisation for a bank-scale deployment. The deep baselines' latencies were not re-measured on the idle machine and are left out rather than reported unfairly.

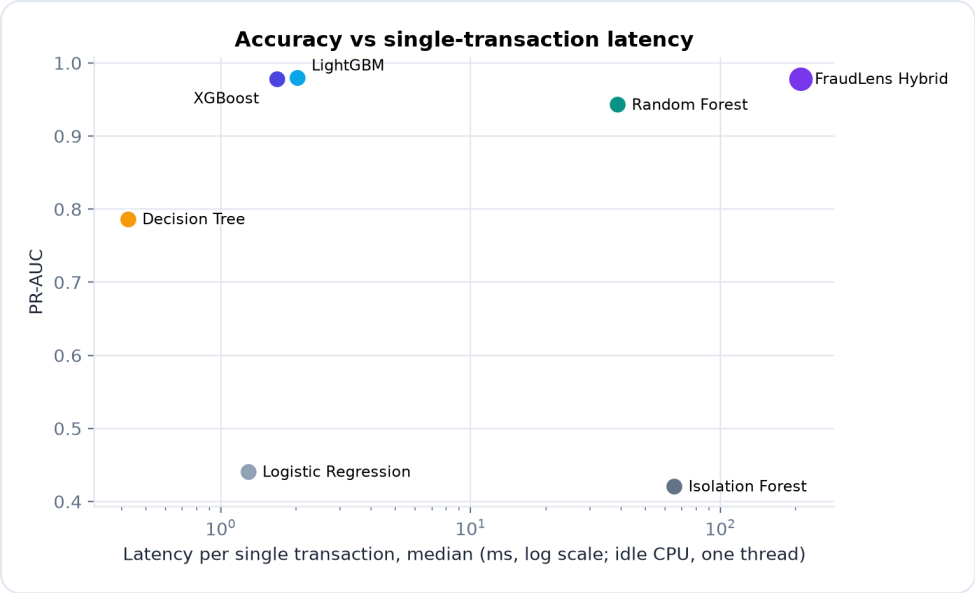


Figure 7. Accuracy versus single-transaction latency (idle machine, one thread). The boosted trees are both fast and accurate; the hybrid's time is dominated by the two scikit-learn forests.

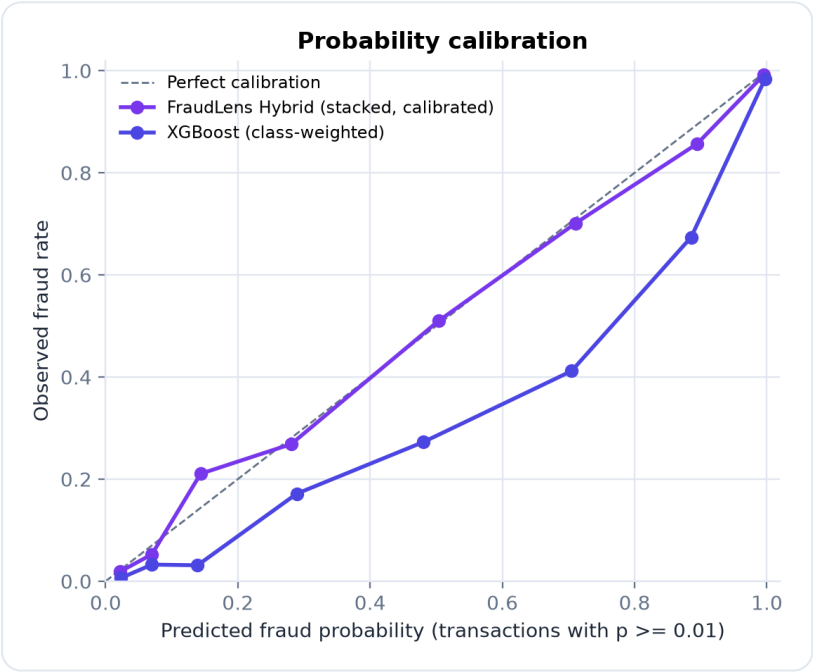


Figure 8. Calibration: where class-weighted XGBoost says 70%, only 41% of transactions are fraud; where the stacked hybrid says 71%, 70% are. The risk score is built on the hybrid's honest probabilities.

7.2 E2: imbalance handling

Table 14. E2: imbalance strategies (actual test fraud rate 0.60%)

Model	Strategy	PR-AUC	Prec @0.5	Recall @0.5	F1 @0.5	F1 tuned	Brier	Mean p
Logistic Regression	No balancing	0.652	0.885	0.443	0.591	0.675	0.00309	0.0049
Logistic Regression	Class weighting	0.441	0.190	0.945	0.316	0.581	0.02464	0.0734
Logistic Regression	Random undersampling (1:1)	0.433	0.179	0.945	0.300	0.580	0.02604	0.0772
Logistic Regression	SMOTE oversampling (1:1)	0.451	0.192	0.940	0.319	0.591	0.02271	0.0658

Model	Strategy	PR-AUC	Prec @0.5	Recall @0.5	F1 @0.5	F1 tuned	Brier	Mean p
Random Forest	No balancing	0.951	0.994	0.745	0.852	0.894	0.00124	0.0057
Random Forest	Class weighting	0.943	0.894	0.856	0.875	0.873	0.00147	0.0109
Random Forest	Random undersampling (1:1)	0.880	0.210	0.974	0.345	0.810	0.02003	0.0726
Random Forest	SMOTE oversampling (1:1)	0.941	0.831	0.897	0.863	0.875	0.00167	0.0124
XGBoost	No balancing	0.971	0.967	0.894	0.929	0.928	0.00069	0.0056
XGBoost	Class weighting	0.978	0.918	0.948	0.932	0.937	0.00066	0.0066
XGBoost	Random undersampling (1:1)	0.960	0.371	0.985	0.539	0.907	0.00785	0.0198
XGBoost	SMOTE oversampling (1:1)	0.980	0.962	0.921	0.941	0.940	0.00055	0.0059

Finding (RQ2)

The best strategy depends on the model: Logistic Regression does best with no balancing (PR-AUC 0.652); Random Forest does best with no balancing (PR-AUC 0.951); XGBoost does best with SMOTE (PR-AUC 0.980). Random undersampling is the worst option for every model and, at the default 0.5 threshold, cuts precision to 18%-37%. Balancing inflates probabilities: class-weighted Logistic Regression predicts an average fraud probability of 0.073 against a true rate of 0.006 (12x). It also shows why ROC-AUC misleads: weighting raises LR's ROC-AUC from 0.970 to 0.990 while its PR-AUC falls from 0.652 to 0.441. For XGBoost, SMOTE and class weighting are within 0.002 PR-AUC of each other; SMOTE needs synthetic data and 1.6x the training time, so the deployed engine keeps class weighting and lets the stacking layer recalibrate the probabilities. The decision threshold matters more than the balancing trick: undersampled XGBoost goes from F1 0.54 at the default 0.5 threshold to 0.91 when the threshold is tuned on validation data.

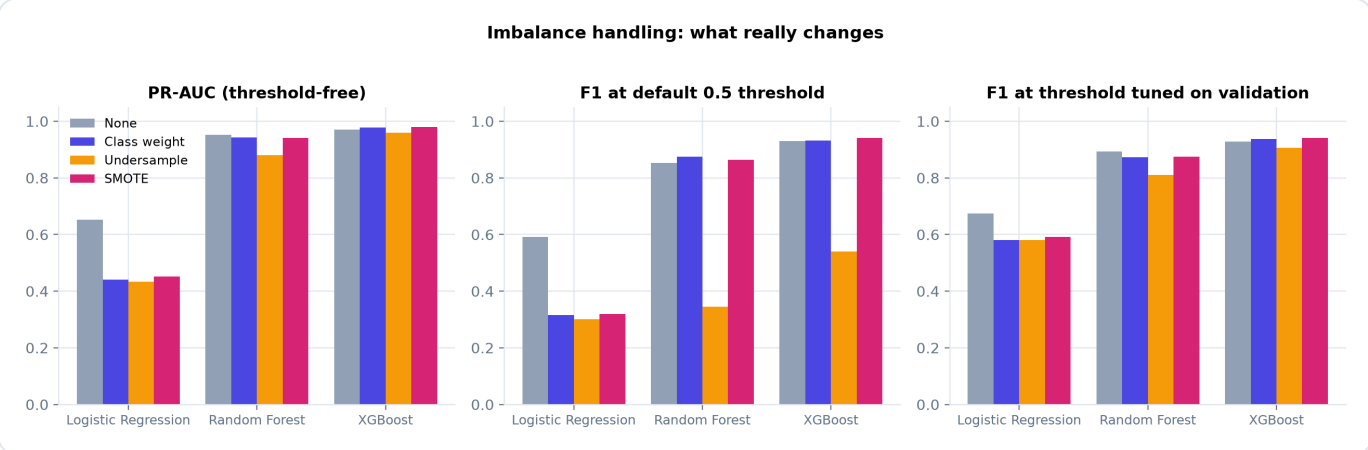


Figure 9. Imbalance strategies: PR-AUC, F1 at the default threshold and F1 at the tuned threshold.

7.3 E3: how evaluation shortcuts inflate results

We trained the same XGBoost model under four protocols, from the shortcut common in published papers to our own protocol:

Table 15. E3: evaluation protocols on the ULB European dataset

Protocol	Accuracy	Precision	Recall	F1	PR-AUC
Predict 'genuine' for everything (no model)	99.87%	0.000	0.000	0.000	0.001

Protocol	Accuracy	Precision	Recall	F1	PR-AUC
SMOTE on the whole dataset, then random 80/20 split, threshold 0.5	99.98%	1.000	1.000	1.000	1.000
Random split first, SMOTE on training data only, threshold 0.5	99.94%	0.796	0.878	0.835	0.883
Chronological split, SMOTE on training data only, threshold tuned on validation	99.95%	0.915	0.720	0.806	0.801
Chronological split, class weighting (no synthetic data), threshold tuned on validation (FraudLens)	**99.95%**	**0.887**	**0.733**	**0.803**	**0.782**

Table 16. E3: evaluation protocols on the Sparkov dataset

Protocol	Accuracy	Precision	Recall	F1	PR-AUC
Predict 'genuine' for everything (no model)	99.40%	0.000	0.000	0.000	0.006
SMOTE on the whole dataset, then random 80/20 split, threshold 0.5	99.97%	1.000	1.000	1.000	1.000
Random split first, SMOTE on training data only, threshold 0.5	99.93%	0.958	0.918	0.938	0.974
Chronological split, SMOTE on training data only, threshold tuned on validation	99.92%	0.954	0.912	0.933	0.978
Chronological split, class weighting (no synthetic data), threshold tuned on validation (FraudLens)	**99.92%**	**0.952**	**0.907**	**0.929**	**0.975**

Finding (RQ3): why many published numbers are not comparable

On ULB, oversampling before splitting (protocol A) reports accuracy 99.98%, precision 100.0% and F1 1.000. These numbers look like the 99.9% results in T3 and T7. The identical model evaluated honestly on future data (protocol D) gives F1 0.803. Meanwhile a model that never flags anything reaches 99.87% accuracy. Synthetic fraud points leak into the test set and make it easy; accuracy hides the minority class. This reproduces the leakage analysis of Santos et al. (2018) and Hayat & Magnier (2025).

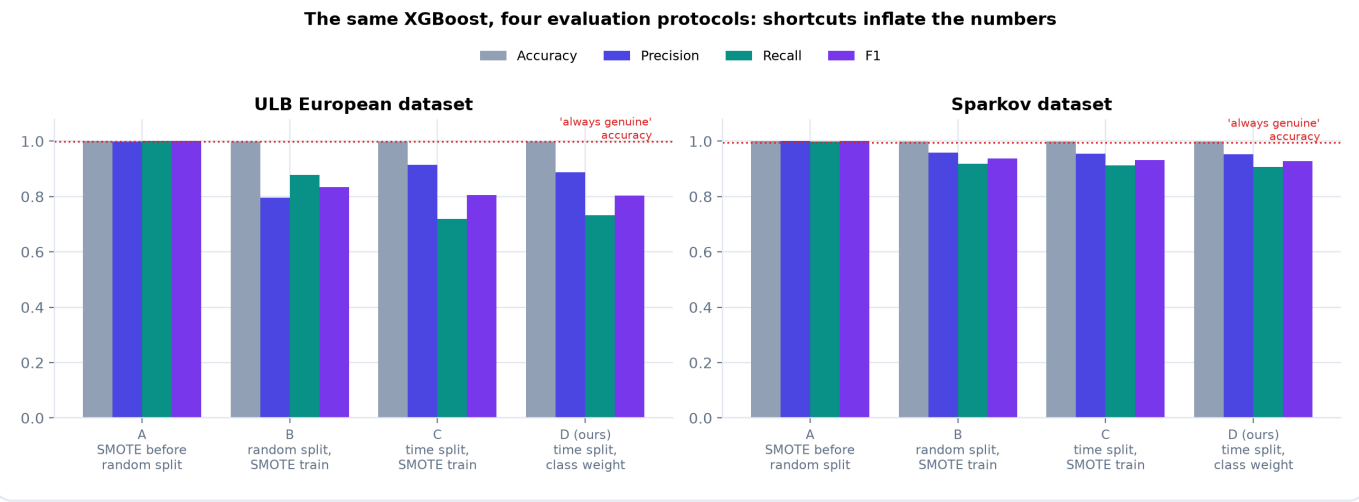


Figure 10. The same XGBoost under four evaluation protocols.

7.4 E4: ablation study

Table 17. E4a: feature engineering (XGBoost, same split)

Feature set	# features	PR-AUC	Recall	Precision	F1	vs FraudLens features (diff [95% CI])
Raw transaction fields only (typical project)	23	0.825	0.759	0.768	0.764	-0.1537 [-0.1718, -0.1358]
+ behavioural features (FraudLens, protected attributes excluded)	**39**	**0.978**	**0.937**	**0.937**	**0.937**	**reference**
+ age and gender (protected attributes)	41	0.984	0.943	0.962	0.952	+0.0057 [+0.0037, +0.0079]

Table 18. E4b: hybrid components

Combination (stacked)	PR-AUC	Recall	Precision	F1	vs XGBoost only (diff [95% CI])
XGBoost only	0.9784	0.936	0.938	0.937	reference
Random Forest only	0.9431	0.852	0.895	0.873	-0.0353 [-0.0431, -0.0290]
Isolation Forest only	0.4210	0.513	0.479	0.496	-0.5574 [-0.5884, -0.5242]
XGBoost + RF	0.9786	0.937	0.936	0.936	+0.0002 [-0.0000, +0.0004]
XGBoost + IF	0.9783	0.928	0.947	0.937	-0.0001 [-0.0003, +0.0001]
RF + IF	0.9429	0.822	0.928	0.872	-0.0355 [-0.0432, -0.0290]
XGBoost + RF + IF (FraudLens)	**0.9785**	**0.925**	**0.946**	**0.935**	**+0.0001 [-0.0002, +0.0005]**
Simple average of XGB and RF	0.9731	0.924	0.943	0.933	-0.0053 [-0.0079, -0.0031]
XGBoost with IF score as an input feature	0.9801	0.936	0.949	0.942	+0.0017 [+0.0001, +0.0032]

Finding (RQ4)

Behavioural features are the single biggest improvement: the same XGBoost moves from PR-AUC 0.825 on raw transaction fields (what typical projects use) to 0.978 (+0.154). Adding age and gender would add +0.0057 PR-AUC; we leave these protected attributes out, trading that small gain for a fairness guarantee. Among hybrid designs, stacking XGBoost + RF + IF matches XGBoost alone (+0.0001, 95% CI [-0.0002, +0.0005]). Feeding the Isolation Forest score into XGBoost as an input feature (Carcillo et al., 2021) gives PR-AUC 0.980, a small but statistically significant gain (+0.0017, 95% CI [+0.0001, +0.0032]), a candidate for the next model version.

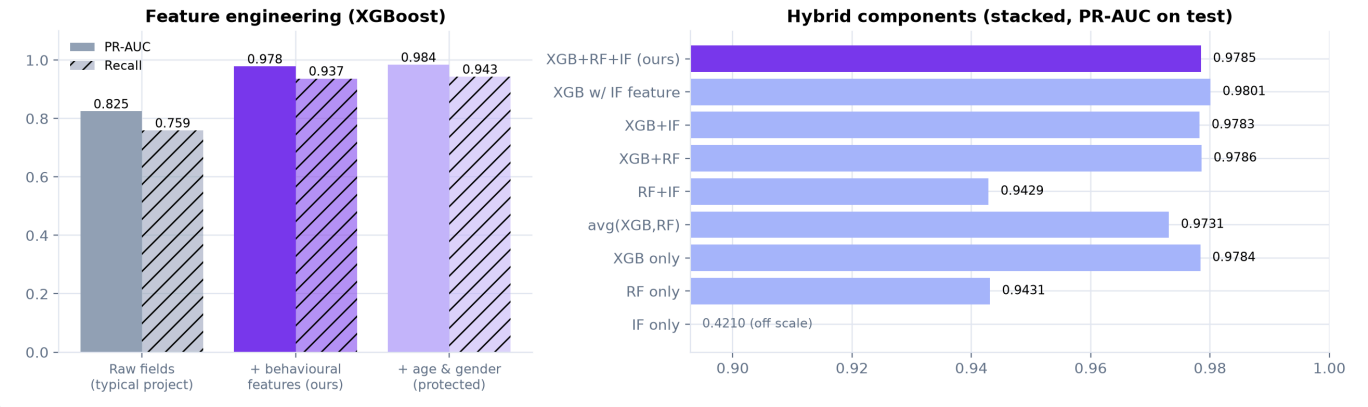


Figure 11. Ablation: feature sets (left) and stacked detector combinations (right).

7.5 E5: catching a fraud type never seen in training

Supervised models only learn fraud that appears in their training labels. To test the anomaly layer we removed one entire fraud type (all online frauds, or all in-person frauds) from training and validation, retrained everything, and measured how many of that type's test frauds each detector still catches at its validation-chosen threshold.

Table 19. E5: recall (precision) on the never-seen fraud type

Held-out type	Test frauds	XGBoost	Random Forest	Isolation Forest	Novelty alarm N1	Hybrid (stacking only)	Hybrid + safety layer	Hybrid that saw it
online	391	0.537 (0.60)	0.000 (0.00)	0.987 (0.06)	0.240 (0.68)	0.432 (0.58)	0.496 (0.51)	0.997 (0.37)
in-person	701	0.227 (0.35)	0.000 (0.00)	0.437 (0.19)	0.023 (0.26)	0.106 (0.30)	0.123 (0.27)	0.976 (0.50)

Finding (RQ5): an honest limit

Holding out online fraud, XGBoost still catches 53.7% of it; the stacking-only hybrid 43.2% and the hybrid with the safety layer 49.6% (+6.4%), against 99.7% for a hybrid that saw this type. Holding out in-person fraud, XGBoost still catches 22.7% of it; the stacking-only hybrid 10.6% and the hybrid with the safety layer 12.3% (+1.7%), against 97.6% for a hybrid that saw this type. The safety layer adds +1.7% to +6.4% recall on the unseen type, a real but modest gain. The stacking layer, fitted on the fraud types it has seen, does worse than XGBoost alone on the new one: a learned combination over-fits to known fraud, which is exactly why the novelty route sits outside it. The Isolation Forest on its own catches 44%-99% of the unseen fraud, but at only 6%-19% precision, so it is a triage signal, not a decision maker. The honest conclusion: fraud that is completely absent from the labels is caught far less often than fraud the system has seen. No detector replaces labels. That is why analyst verdicts from the case queue are stored as labels for retraining, so a new fraud type only goes undetected until it has been labelled.

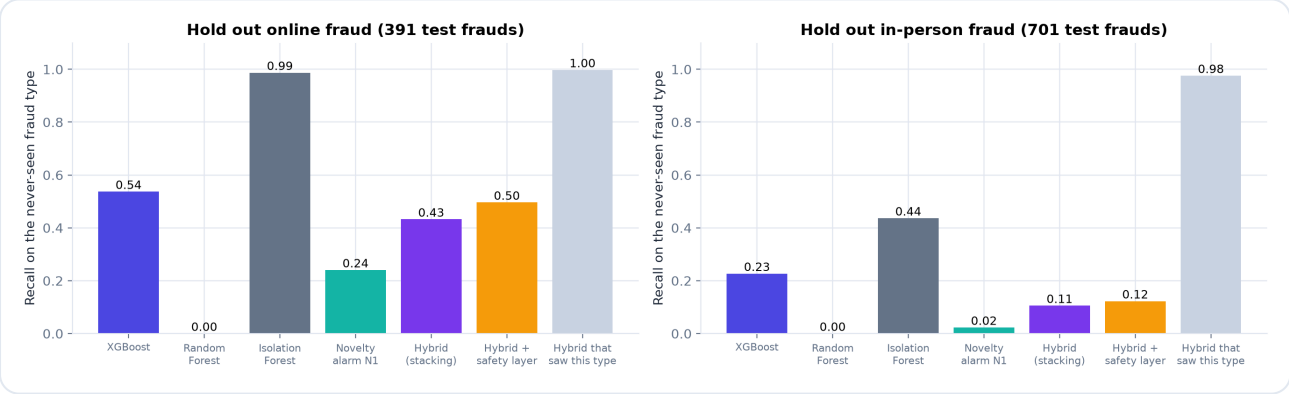


Figure 12. Recall on a fraud type removed from training.

7.6 E6: ULB European benchmark

Table 20. E6: ULB dataset, last 20% of the timeline as test

Model	PR-AUC [95% CI]	ROC-AUC	Precision	Recall	F1	MCC
FraudLens Hybrid (XGB + RF + IF)	**0.795 [0.694-0.868]**	**0.978**	**0.905**	**0.760**	**0.826**	**0.829**
XGBoost	0.793 [0.693-0.867]	0.983	0.905	0.760	0.826	0.829
LightGBM	0.792 [0.688-0.866]	0.963	0.851	0.760	0.803	0.804
Random Forest	0.808 [0.710-0.885]	0.951	0.853	0.773	0.811	0.812

Model	PR-AUC [95% CI]	ROC-AUC	Precision	Recall	F1	MCC
Deep MLP (PyTorch)	0.809 [0.715-0.883]	0.984	0.931	0.720	0.812	0.819
Logistic Regression	0.778 [0.678-0.854]	0.984	0.911	0.680	0.779	0.787
Isolation Forest (unsupervised)	0.051 [0.036-0.073]	0.954	0.084	0.560	0.146	0.214

ULB has only 75 frauds in the test slice, so the confidence intervals are wide and the models overlap. Because the features are anonymous PCA components, behavioural features and readable reasons are impossible here, which is why Sparkov is our primary dataset.

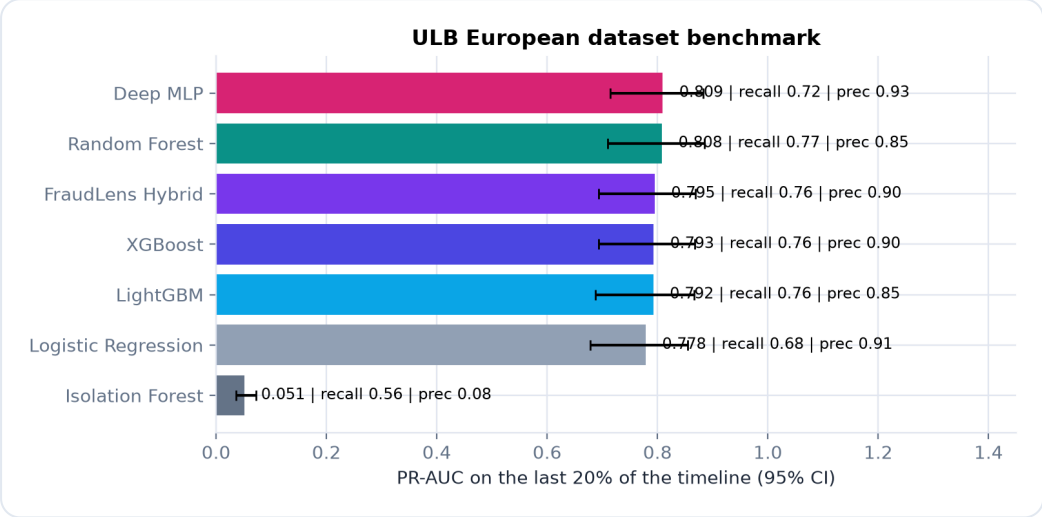


Figure 13. ULB benchmark with 95% confidence intervals.

7.7 E7: business impact, risk tiers and stability

Table 21. E7: cost-optimal thresholds for different alert costs (test period, 75 days)

Alert cost	Threshold	Recall	Precision	Alerts/day	Fraud $ stopped	Total cost	Loss avoided
$1	0.0045	0.987	0.454	31.8	$582,802	$3,706	99.4%
$2	0.0045	0.987	0.454	31.8	$582,802	$6,079	99.0%
$5	**0.0142**	**0.984**	**0.614**	**23.5**	**$581,969**	**$10,917**	**98.1%**
$10	0.0840	0.973	0.830	17.2	$578,876	$18,060	96.9%
$25	0.0874	0.973	0.836	17.1	$578,876	$37,035	93.7%

Table 22. Risk tiers on the test period

Tier	Transactions	Share	Per day	Frauds	Fraud rate	Fraud $
LOW	181,024	99.04%	2429.6	18	0.0%	$2,167
MEDIUM	599	0.33%	8.0	36	6.0%	$13,891
HIGH	1,151	0.63%	15.4	1,038	90.2%	$568,077

Finding (RQ6)

With a $5 alert cost, the hybrid stops 99.6% of fraud value and avoids 98.1% of fraud losses after paying for every alert, raising about 23 alerts per day out of ~2,453 daily transactions. The legacy rule engine avoids only 85.4%. Auto-block (HIGH) is reserved for 0.63% of transactions with a fraud rate of 90.2%.

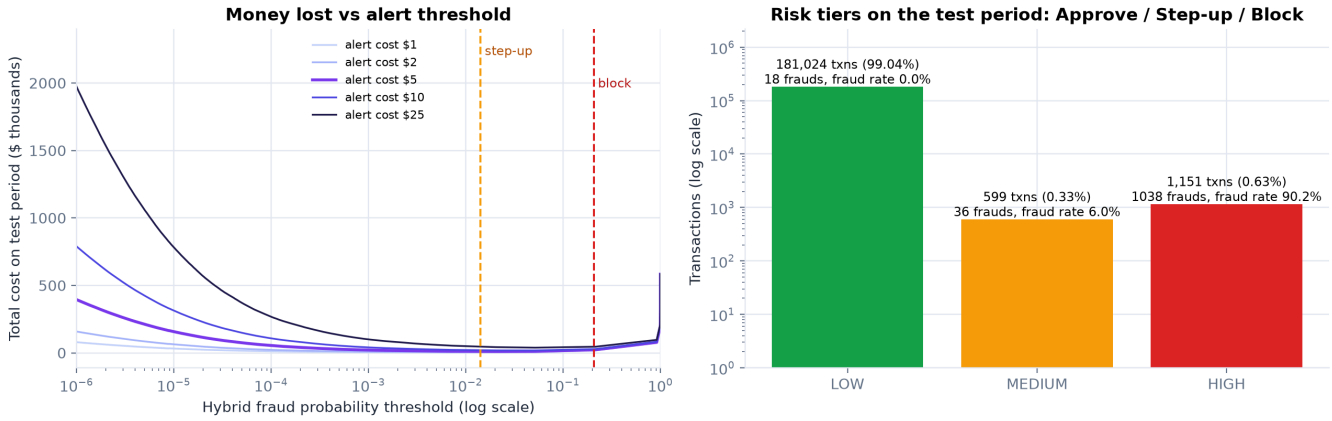


Figure 14. Cost versus threshold (left) and the three risk tiers (right).

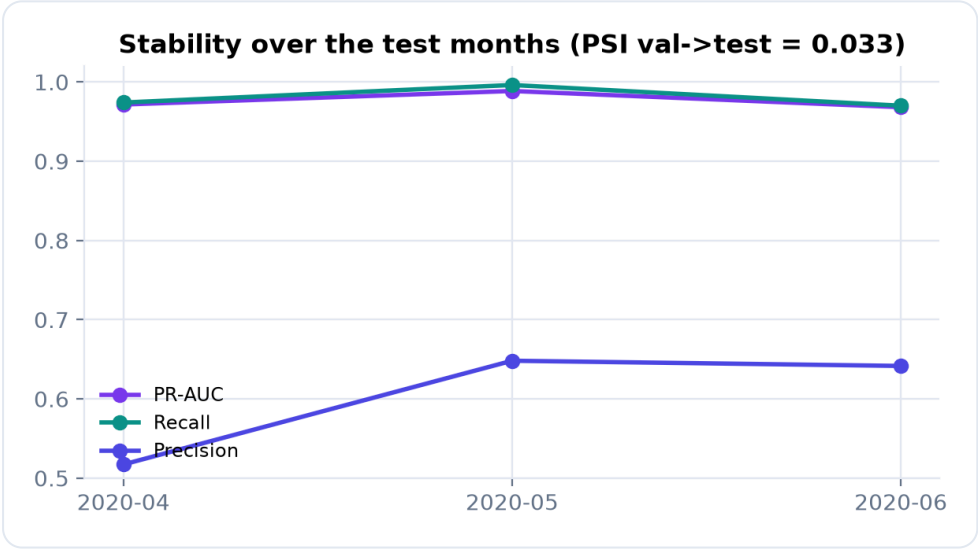


Figure 15. Month-by-month stability on the test period.

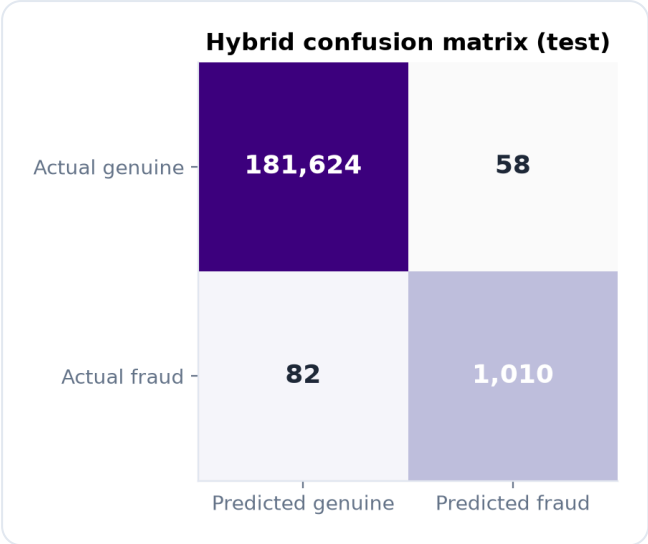


Figure 16. Confusion matrix of the hybrid at the validation-tuned threshold.

The Population Stability Index of the risk score between validation and test is 0.033 (below 0.1 is conventionally "stable"). In production the same statistic, computed weekly, triggers retraining when fraud patterns drift.

7.8 Safety layer: cost and effect

Table 23. How often each rule fires, and what it adds beyond the ML's own alerts

Period	Rule	Fires	On genuine	Fraud rate	Extra alerts	Extra frauds	Extra / day
Validation	P1	49	5	90%	5	0	0.06
Validation	P2	0	0	-	0	0	0.00
Validation	P3	6	6	0%	6	0	0.07
Validation	N1	137	29	79%	22	0	0.26
Test	P1	52	9	83%	8	0	0.11
Test	P2	0	0	-	0	0	0.00
Test	P3	15	15	0%	15	0	0.20
Test	N1	155	45	71%	39	0	0.52

Table 24. The same test period with and without the safety layer

At the alert (step-up) level, test period	ML only	ML + safety layer
Alerts	1,750	1,810
Frauds caught	1,074 / 1,092	1,074 / 1,092
Recall	98.4%	98.4%
Precision	61.4%	59.3%
Fraud losses avoided (net of alert costs)	98.1%	98.1%

How we prove it is worth keeping

On real, unseen traffic the safety layer costs 0.81 extra step-ups per day, leaves recall unchanged at 98.4%, and lowers alert precision from 61.4% to 59.3%. It adds no fraud here because the ML already catches the fraud types present in Sparkov. It is insurance for attacks absent from the training data (card testing and cloned cards, Section 8.1), bought at a measured, small price.

7.9 Explainability results

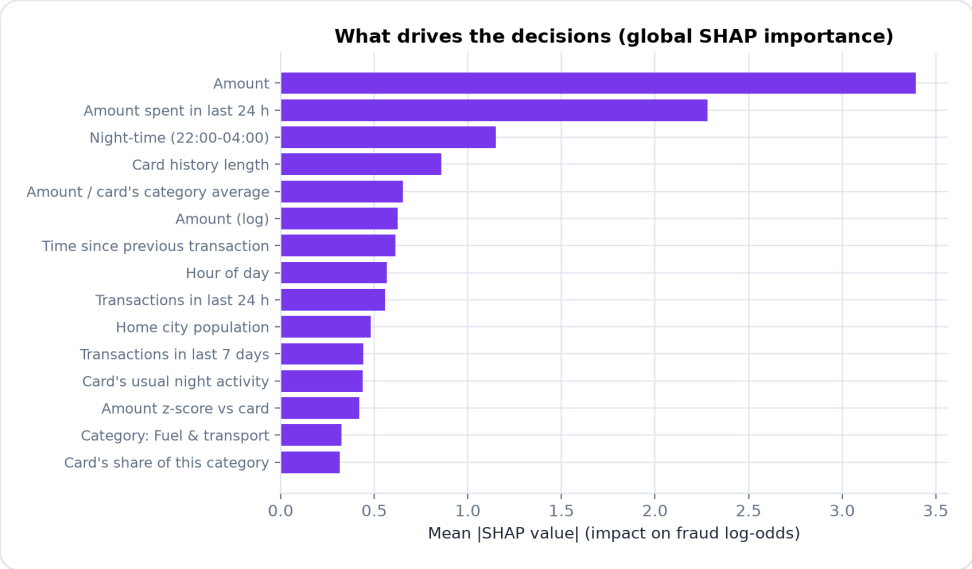


Figure 17. Global SHAP importance: which signals drive decisions overall.

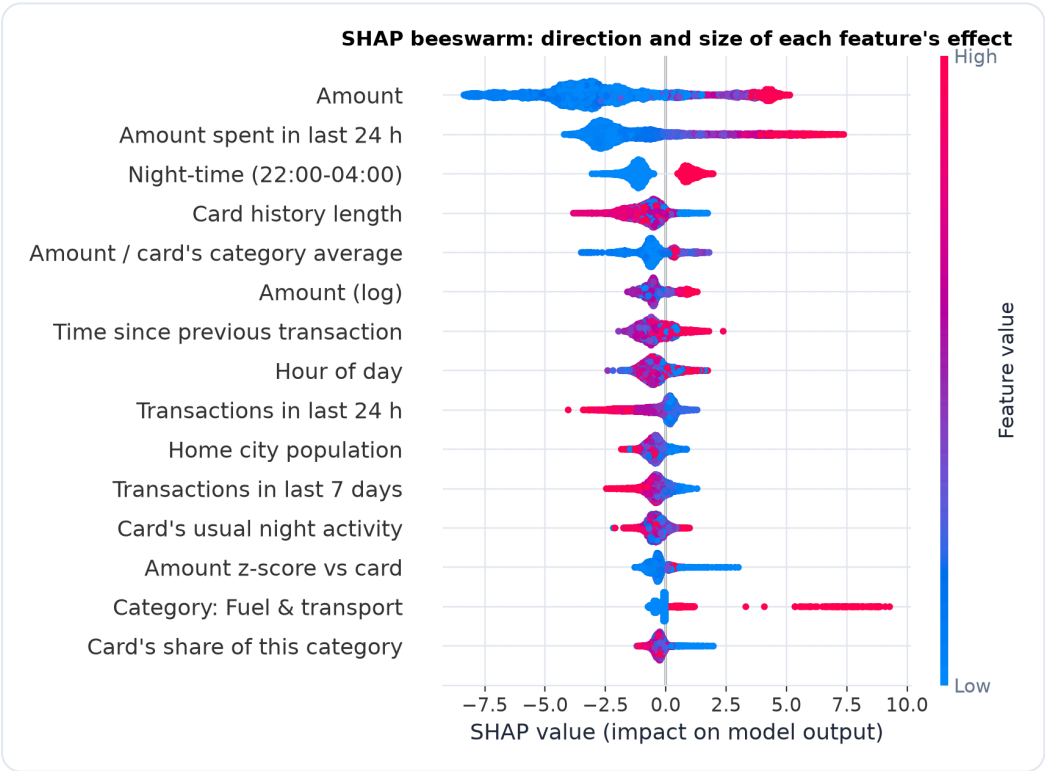


Figure 18. SHAP beeswarm: each dot is one transaction; red = high feature value; right = towards fraud.

Table 25. Case studies from the test period (including a false alarm and a missed fraud, for honesty)

Case	Transaction	Risk	Decision	Top reasons given by FraudLens
fraud caught (HIGH risk)	$514.17 · Entertainment · 05-03 03:03	81 (HIGH)	fraud	Amount $514.17 is 6.2x this card's average spend ($83); Made late at night (03:00-04:00); only 23% of this card's past transactions were at night; Entertainment has a higher fraud rate in historical data
fraud caught (HIGH risk)	$326.23 · Grocery (in-store) · 05-13 09:51	100 (HIGH)	fraud	Grocery (in-store) has a higher fraud rate in historical data; Large transaction amount ($326.23); 2 transactions ($1,906) in the past 24 h, 0.9x this card's usual daily activity
fraud caught (HIGH risk)	$988.90 · Shopping (in-store) · 04-10 23:58	100 (HIGH)	fraud	6 transactions ($3,069) in the past 24 h, 3.2x this card's usual daily activity; Amount $988.90 is 13.3x this card's average spend ($74); Made late at night (23:00-00:00); only 28% of this card's past transactions were at night
fraud caught (HIGH risk)	$49.76 · Miscellaneous (in-store) · 06-18 22:23	92 (HIGH)	fraud	Made late at night (22:00-23:00); only 27% of this card's past transactions were at night; 2 transactions ($640) in the past 24 h, 0.7x this card's usual daily activity; Miscellaneous (in-store) has a higher fraud rate in historical data
fraud sent to step-up verification (MEDIUM)	$19.91 · Personal care · 05-12 23:35	47 (MEDIUM)	fraud	Made late at night (23:00-00:00); only 22% of this card's past transactions were at night; Large transaction amount ($19.91)
false alarm (genuine blocked)	$1,072.30 · Online shopping · 05-30 23:23	100 (HIGH)	genuine	Amount $1,072.30 is 11.7x this card's average spend ($92); 7 transactions ($2,421) in the past 24 h, 1.5x this card's usual daily activity; 9.4x what this card usually spends on Online shopping
missed fraud (approved)	$780.14 · Shopping (in-store) · 06-15 18:00	0 (LOW)	fraud	Amount $780.14 is 13.6x this card's average spend ($57); 9.0x what this card usually spends on Shopping (in-store)
normal transaction (approved)	$1.20 · Online miscellaneous · 05-17 09:19	0 (LOW)	genuine	2 transactions ($478) in the past 24 h, 1.0x this card's usual daily activity

8 How FraudLens detects scams

Different scams leave different fingerprints in transaction data. The table below shows which signals FraudLens uses for each type and which detector is responsible; Section 8.1 shows the system's actual responses when each scam is replayed against a real card's history.

Table 26. Scam typology and detection signals

Scam / fraud type	How it works	Tell-tale signals	FraudLens features	Detector
Card-not-present fraud (stolen card details)	Card data from phishing, breaches or malware is used online	Online channel, large amount, night-time, unfamiliar category	is_online, cat_*, amt_to_card_mean, is_night, is_new_category	XGBoost + RF
Card testing (enumeration)	Tiny charges check which stolen numbers work, then a large purchase	Burst of small online charges, then a jump	cnt_1h, log_secs_since_last, amt_to_card_mean	Safety layer: P3 micro-charge burst, then P1 + N1 (absent from training data)
Stolen-card spree / account takeover	Maximum spending in a few hours before the card is blocked	24 h count and spend far above normal	cnt_24h_vs_usual, sum_24h, amt_zscore	XGBoost + RF
Cloned card (skimming) / impossible travel	A copy of the card is used far away	Distance/speed since the previous transaction	dist_prev_km, speed_kmh, dist_home_km	Safety layer: P2 impossible travel (geography is not learnable from Sparkov)
Limit probing / escalation	Rising amounts test the card's limit	Growing 24 h spend, amount vs history	sum_24h, amt_to_cat_mean	XGBoost
Dormant-card reactivation	An unused card suddenly spends	Long gap then a high amount	log_secs_since_last, amt_to_card_mean	XGBoost + IF
Merchant compromise / collusion	Fraud concentrates at a merchant	Merchant-level fraud rates, card-merchant links	Not yet (future: merchant risk, graph model)	Future work
Social engineering / authorised push payment (e.g. OTP or UPI scams)	The real customer is tricked into paying	New payee, unusual amount, device/session anomalies	Partly (amount, novelty); needs device and payee data	Limitation

8.1 Scam scenarios replayed through the live engine

Table 27. Scam scenarios played against a real card's history (sandboxed): ML alone vs final decision

Scenario	Steps	ML risk per step	ML alone	Final decision	Rules fired	Top reasons
Card-testing attack	5	0 -> 0 -> 0 -> 0 -> 28	LOW	MEDIUM at step 4	N1, P1, P3	Policy P3: tiny online charge ($3.10) after 3 others in the past hour, the classic card-testing pattern; Made late at night (23:00-00:00); only 19% of this card's past transactions were at night
Stolen-card spending spree	6	100 -> 13 -> 0 -> 99 -> 100 -> 100	HIGH	HIGH at step 1	N1	Grocery (in-store) has a higher fraud rate in historical data; Amount $312.40 is 6.3x this card's average spend ($50)

Scenario	Steps	ML risk per step	ML alone	Final decision	Rules fired	Top reasons
Late-night high-value online purchase	1	81	HIGH	HIGH at step 1	-	Amount $1,249.00 is 25.2x this card's average spend ($50); Made late at night (02:00-03:00); only 19% of this card's past transactions were at night
Impossible travel (cloned card)	2	0 -> 0	LOW	MEDIUM at step 2	P2	Policy P2: 1,074 km from the previous transaction; the implied speed is faster than a commercial flight; Amount $389.00 is 7.8x this card's average spend ($50)
Escalating amounts (limit probing)	4	0 -> 0 -> 1 -> 62	HIGH	HIGH at step 4	-	Amount $690.00 is 13.9x this card's average spend ($50); Made late at night (22:00-23:00); only 19% of this card's past transactions were at night
Normal customer day (control)	3	0 -> 0 -> 0	LOW	LOW	-	Fuel & transport has a higher fraud rate in historical data

Each scenario starts from the real history of a long-standing, mostly-daytime cardholder and runs in a sandbox copy of that card's state, so the demo never changes live data. The ML alone catches the fraud styles present in its training data (stolen-card spending spree, late-night high-value online purchase, escalating amounts (limit probing)). The ML alone approves every step of card testing (the pattern never occurs in the training data); the safety layer escalates it at step 4 of 5. The ML alone approves every step of cloned-card travel (the pattern never occurs in the training data); the safety layer escalates it at step 2 of 2. The normal customer day is approved at every step. The same scenarios run live from the dashboard's Scam Scenarios page.

Limitation

Transaction-level models cannot see everything. Scams where the genuine customer is persuaded to pay (authorised push payment / social engineering) look like normal customer behaviour and need extra signals such as device fingerprint, payee history and session behaviour. Merchant-level collusion needs a graph view (T14). Both are listed as future work rather than claimed.

9 What we do better, and how we prove it

9.1 Compared with typical existing work

Table 28. FraudLens versus typical existing work

Aspect	Typical paper / project	FraudLens	Evidence
Model	One classifier	Stacked XGB + RF + Isolation Forest	E1, E4
Unknown fraud	Not addressed	Novelty alarm + documented policy rules outside the stacking	E5, 7.8, 8.1
Features	Raw columns or PCA	Per-card behavioural history, causal	E4: +0.154 PR-AUC
Imbalance	SMOTE assumed	Four strategies compared; calibrated weighting	E2
Evaluation	Random split, accuracy, SMOTE before split	Chronological split, PR-AUC/MCC/money, bootstrap CIs	E3
Output	Fraud / not fraud	Calibrated probability, 0-100 score, 3 action tiers	E7
Explanation	None or a global chart	Per-decision reasons + mitigating factors + audit record	7.9
Deployment	Notebook	Real-time API, online feature store, dashboard, Docker	Ch. 10
Security & fairness	Rarely discussed	Tokenisation, audit trail, API key, protected attributes excluded	4.2, E4

9.2 Claims and proofs

Table 29. Every claim is backed by a test

Claim	How we test it	Result	Verdict
Behavioural features beat raw fields	E4a, same XGBoost, paired bootstrap	PR-AUC 0.825 -> 0.978	Proven
Trees beat deep learning on this data	E1 on the same split	XGB 0.978 vs MLP 0.960, CNN 0.933	Proven
ML beats legacy rules	E1	PR-AUC 0.979 vs 0.248	Proven
Hybrid is at least as good as its best member	E4b + paired bootstrap vs XGBoost	diff +0.0001 [-0.0002, +0.0005]	Equal (no loss)
Shortcuts inflate published numbers	E3, four protocols	F1 1.000 (leaky) vs 0.803 (honest)	Proven
Probabilities are calibrated	Calibration curve, Brier score	Hybrid tracks the diagonal	Proven (fig.)
Sub-second real-time scoring	Idle-machine latency, one thread, one transaction per call	models p50 209.4 ms; full API path p50 256.6 ms (p95 526.7 ms)	Proven
Safety layer catches attacks absent from training at low cost	Scam replays + cost on validation and test	0.81 extra step-ups/day; card testing and cloned card escalated	Proven

Claim	How we test it	Result	Verdict
No training/serving skew	Parity unit test	Online features = offline features (3,000 txns)	Proven (test)
Explanations are faithful	TreeSHAP local accuracy unit test	Contributions sum to the model's log-odds (same trees)	Proven (test)
Stable over time	Monthly metrics + PSI	PSI 0.033	Proven

9.3 Improvements over our original proposal

Table 30. From proposal to final system

Original plan	What we did	Why
SMOTE or class weighting	Measured four strategies; chose weighting + calibrated stacking	Evidence (E2) and calibration research
Combine XGB, RF and IF somehow	Learned logistic stacking on a later time period	Weights from data; calibrated output
Fixed risk bands 0-30 / 31-60 / 61-100	Kept the bands, anchored to cost-optimal and 90%-precision thresholds	Bands now mean something in money and precision
SHAP explanation	Exact TreeSHAP turned into sentences + mitigating factors	Analysts and customers need words, not plots
FastAPI + dashboard	Plus online feature store, stream replay, scenario sandbox, case queue, audit log	Operable and demonstrable system
PostgreSQL	SQLite by default; PostgreSQL by setting FRAUDLENS_DB_URL	Zero-setup demo, production-ready path
Evaluate with precision/recall/F1/ROC/PR-AUC	Plus MCC, money saved, bootstrap CIs, paired tests, chronological split	Results that survive scrutiny
Isolation Forest inside the hybrid	Kept in the stack, plus its own novelty-alarm route and three policy rules	Stacking gave it almost no weight; attacks absent from training still need coverage
-	Added: leakage demonstration, unseen-fraud test, DL baselines, ULB benchmark, fairness ablation, idle-machine latency	Answer the questions a reviewer will ask

10 Implementation

10.1 Code structure

Table 31. Repository layout

Path	Purpose
fraudlens/features.py	Causal feature store shared by training and the API
fraudlens/hybrid.py	Hybrid risk engine: detectors, stacking, thresholds, risk score
fraudlens/explain.py	TreeSHAP + reason-code generation
fraudlens/models.py, deep.py, rules.py	Model factory, PyTorch baselines, legacy rules
fraudlens/metrics.py	PR-AUC, MCC, cost, thresholds, bootstrap tests
fraudlens/pipeline.py	Experiments E1-E7, artifacts, scenarios (one command)
fraudlens/plots.py	All figures
fraudlens/api/	FastAPI app, scoring service, decision log, scam scenarios
frontend/	React dashboard
reporting/	Generates these Word and PDF reports from the results
tests/	Unit, parity and API tests
artifacts/	Trained engine, feature-store snapshot, held-out test stream, model card
reports/results, reports/figures	Experiment outputs (JSON) and charts (PNG)

10.2 REST API

Table 32. Main API endpoints (full reference: docs/API.md)

Method	Endpoint	Purpose
POST	/api/score	Score and explain one transaction (what-if or committed)
POST	/api/stream/next?n=	Replay the next n unseen test transactions
GET	/api/stats	Live KPIs: precision, recall, fraud $ stopped, latency, histograms
GET	/api/transactions	Decision log with filters (level, source, review status)
POST	/api/transactions/{id}/review	Analyst verdict: confirm fraud / mark genuine
GET / POST	/api/scenarios, /api/scenarios/{id}/run	Scam scenarios in a sandbox
GET	/api/cards, /api/cards/{token}	Tokenised cards and behaviour profiles
GET	/api/model, /api/results/{name}	Model card and research results
GET	/api/health, /docs	Health check; interactive Swagger documentation

10.3 Web dashboard

Table 33. Dashboard pages

Page	What it does
Overview	Project summary, headline metrics, transaction flow
Live Monitor	Replays the unseen test period in time order; live KPIs, feed, charts; click any row for its explanation
Transaction Tester	Pick a real card, craft a transaction, see the risk gauge, detector votes and reasons
Scam Scenarios	One-click card testing, spree, midnight online, impossible travel, escalation, normal day
Case Queue	Alerts awaiting review; confirm or dismiss; forensic audit record
Results	All experiments E1-E7 with tables, charts and figures
Project Report	Progress, milestones, work distribution and deliverables for reviewers (edited via project.json)
About	Architecture, technology stack, how to run

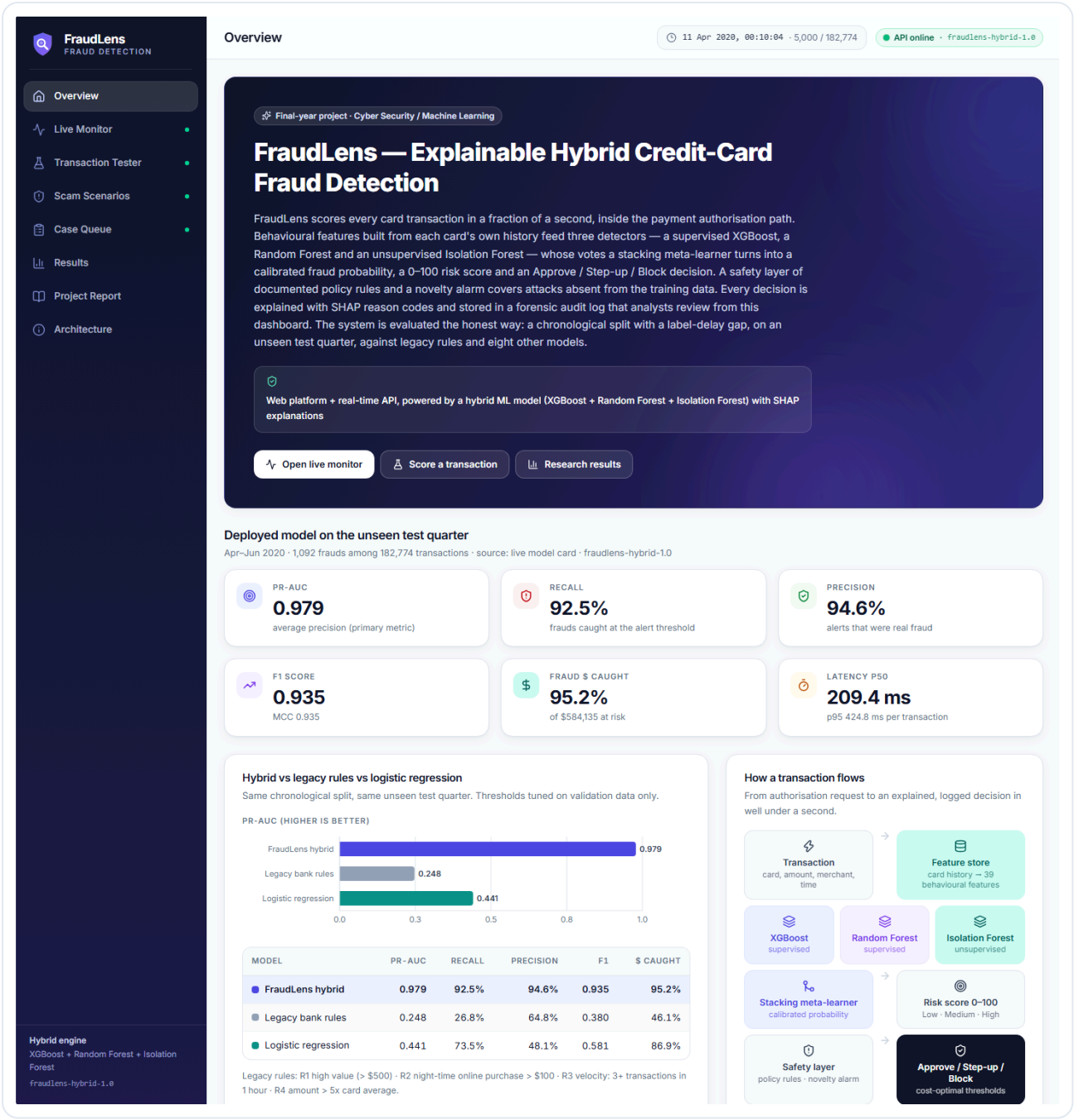


Figure 19. Overview: headline metrics of the deployed model and the transaction flow.

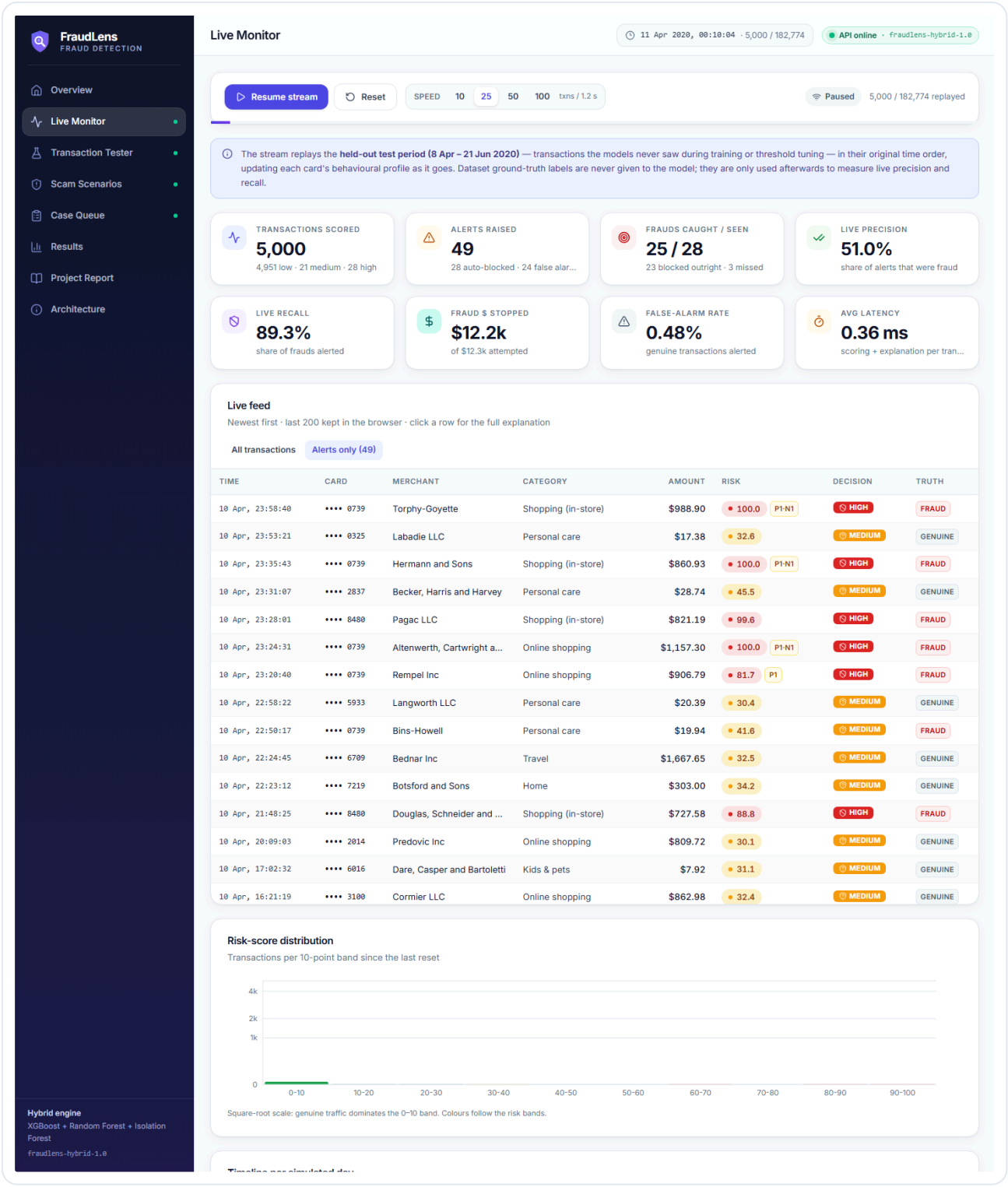


Figure 20. Live Monitor (alerts view): the unseen test quarter replayed through the API, with live precision, recall and money stopped.

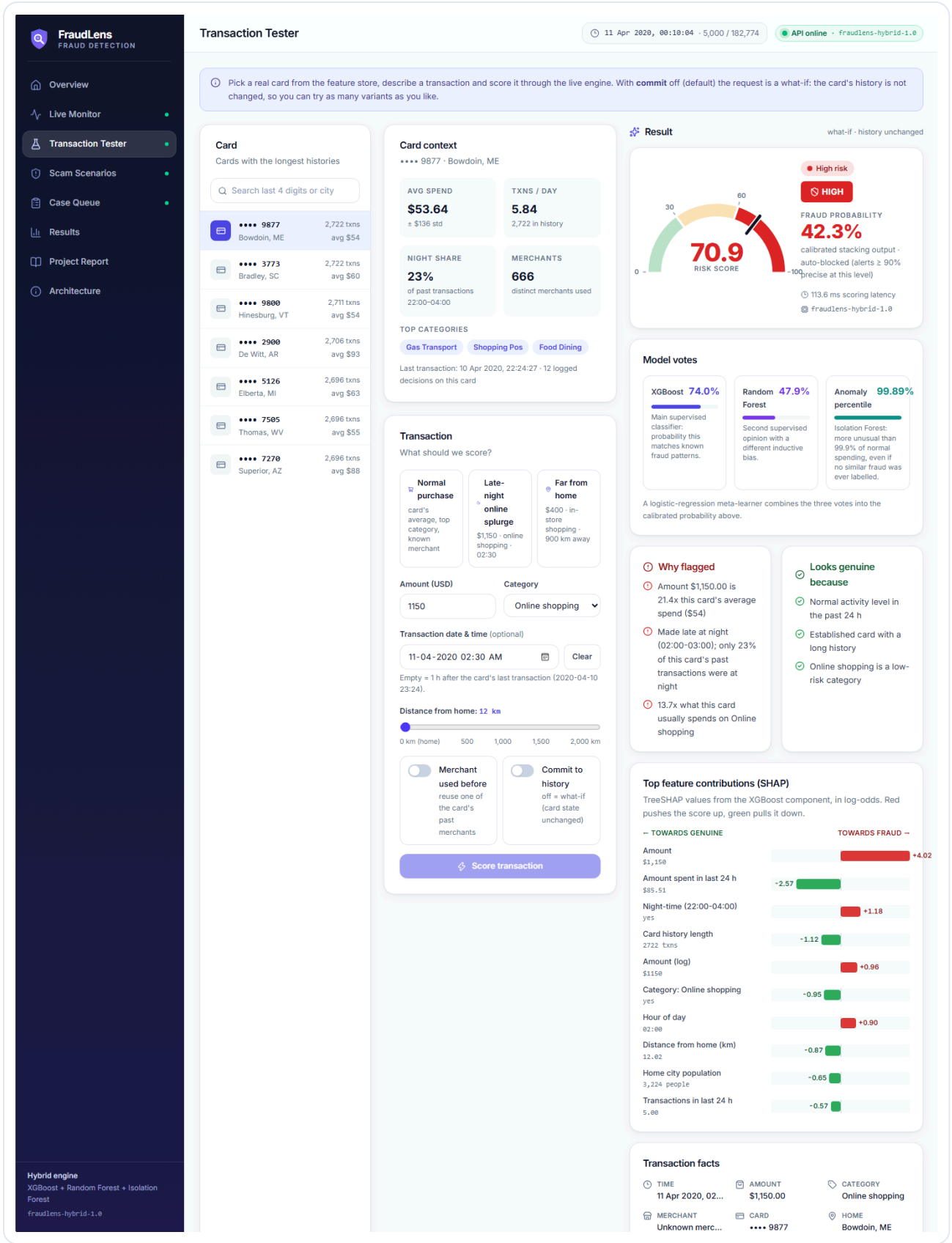


Figure 21. Transaction Tester: a real card, a late-night $1,150 online purchase, the risk gauge, detector votes and SHAP reasons.

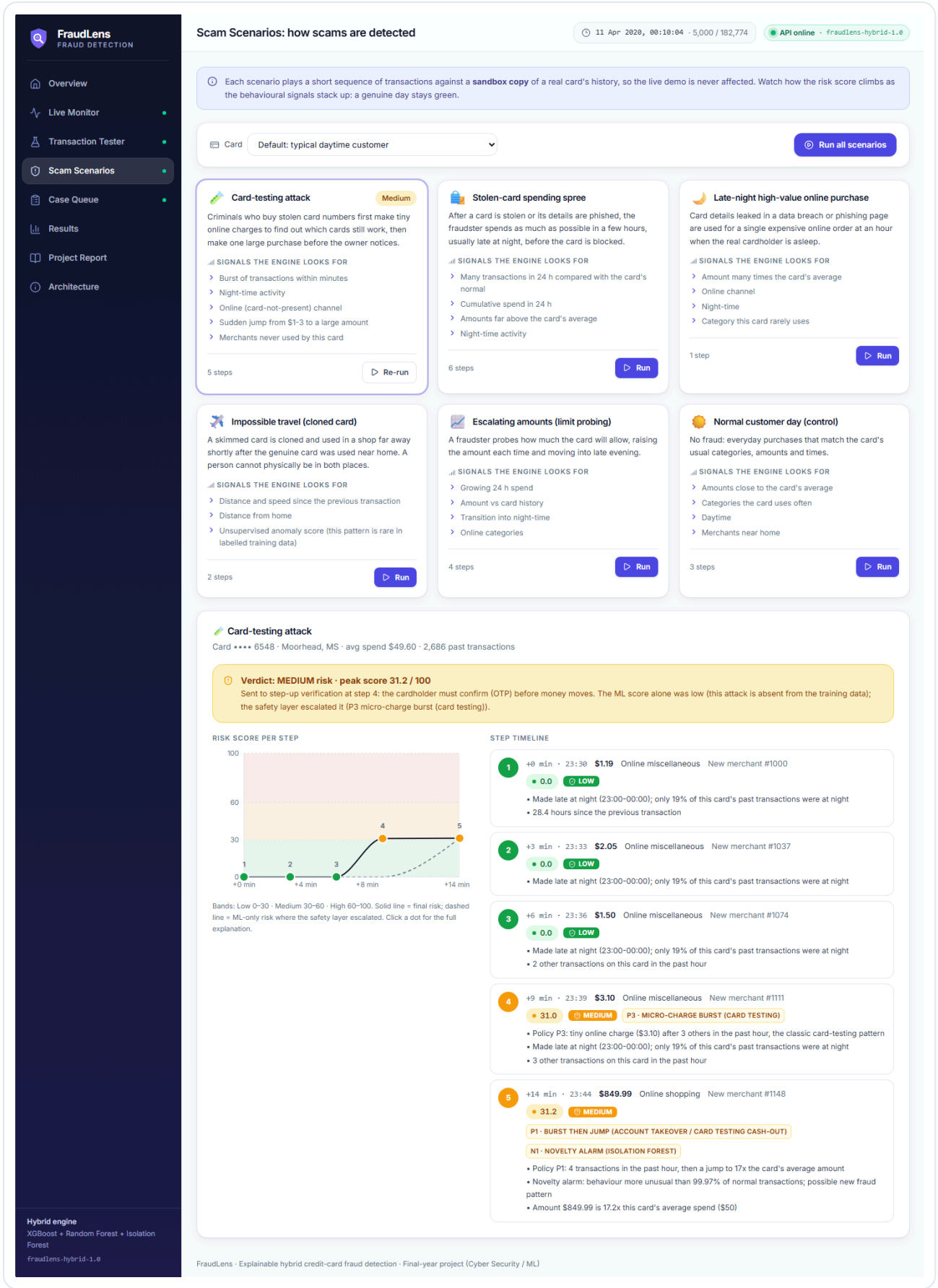


Figure 22. Scam Scenarios: card testing escalated at step 4 by the safety layer while the ML-only risk (dashed) stays flat.

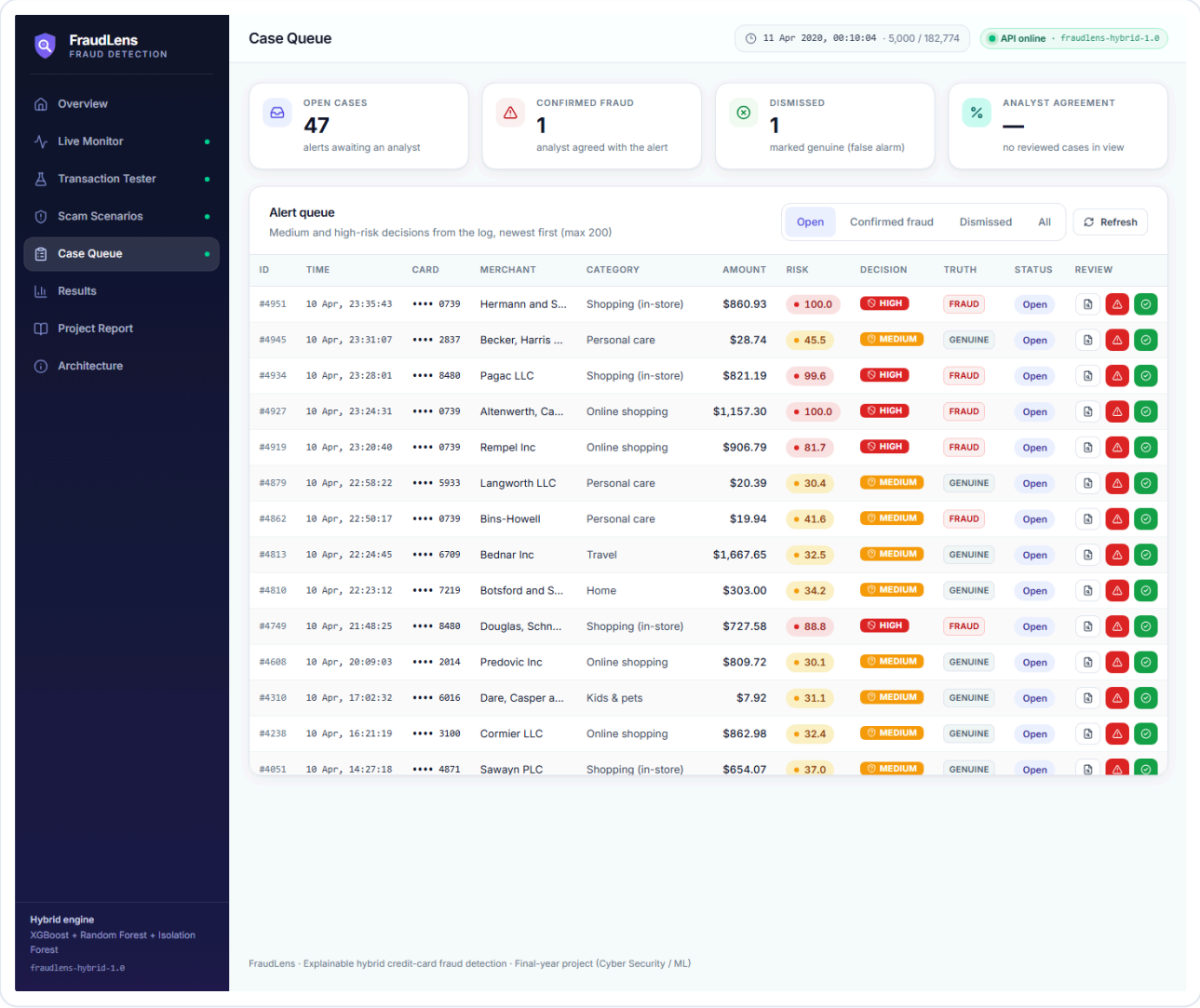


Figure 23. Case Queue: medium and high alerts awaiting analyst review, with the dataset truth for evaluation.

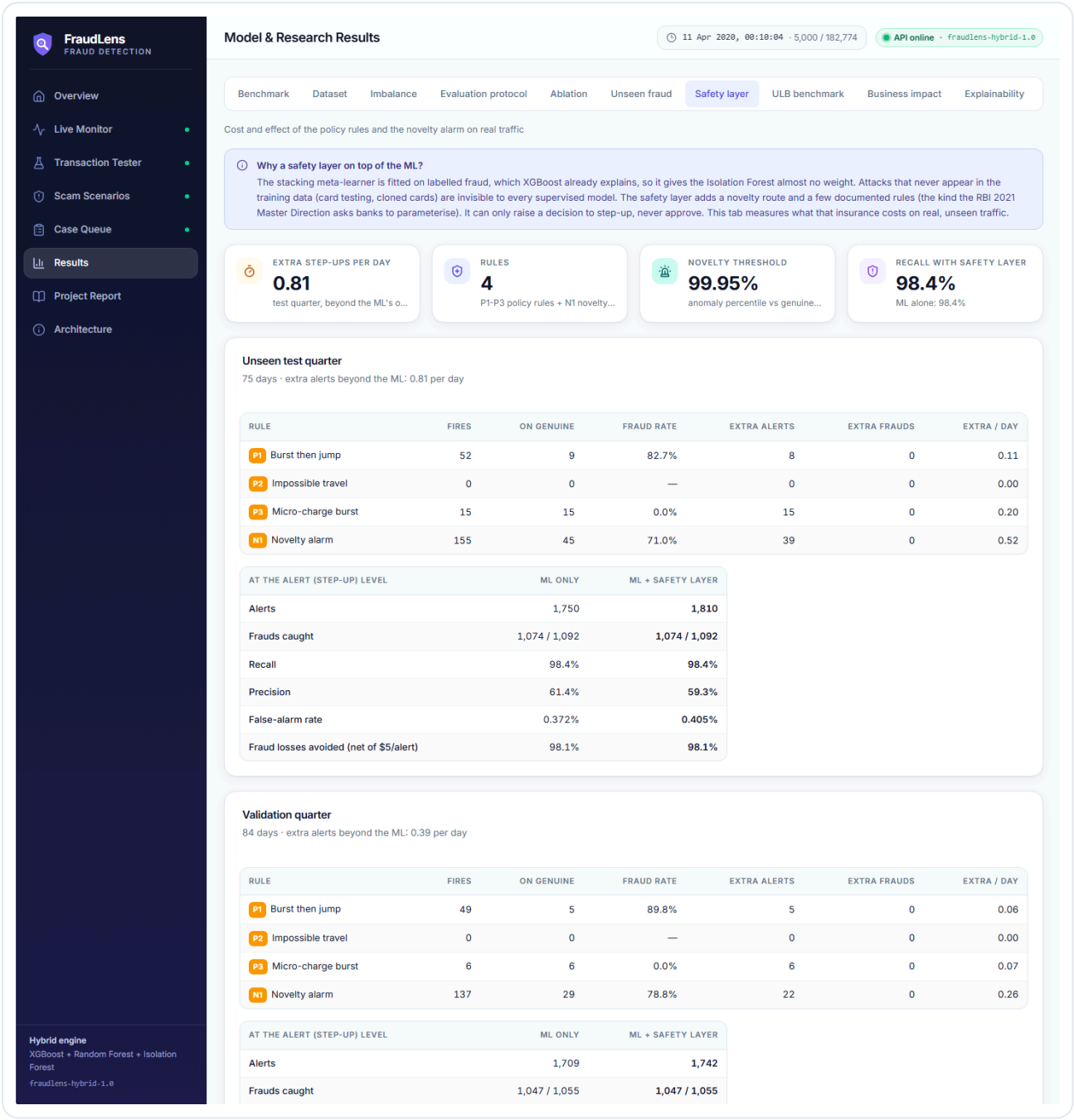


Figure 24. Results, safety-layer tab: what each rule costs and adds on real traffic.

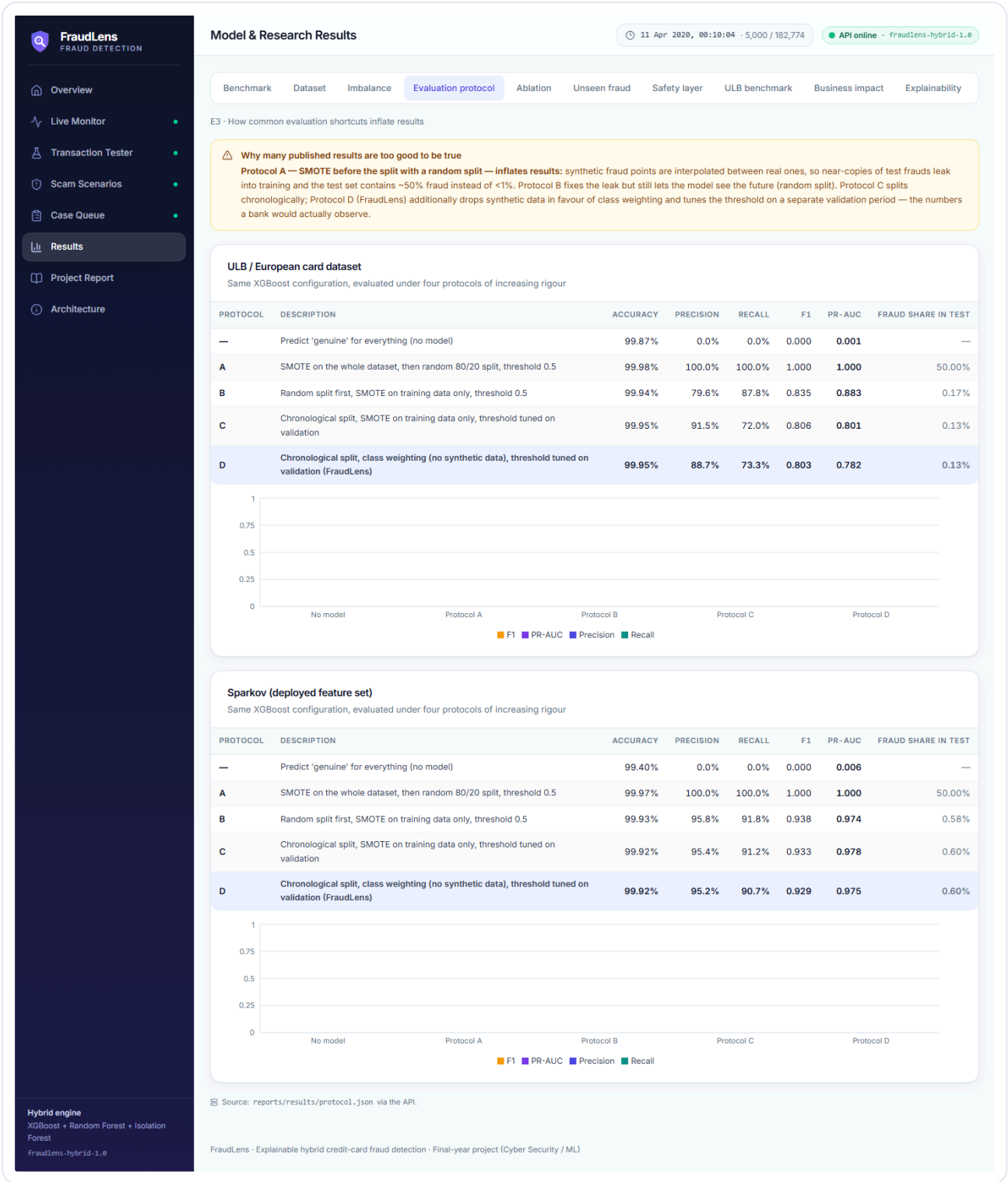


Figure 25. Results, evaluation-protocol tab: how shortcuts inflate published numbers.

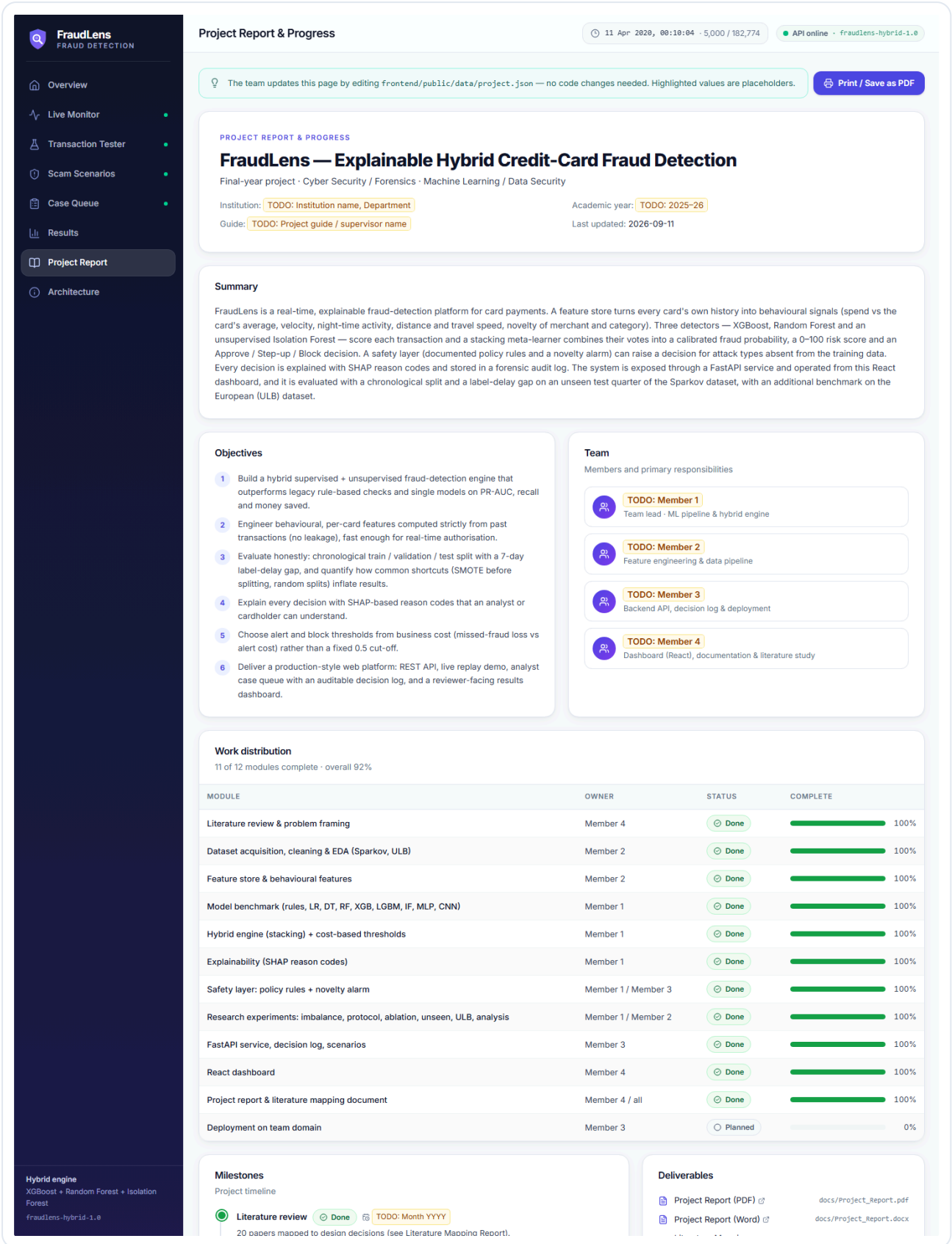


Figure 26. Project Report page for reviewers (edited through project.json).

10.4 Testing

Table 34. Automated tests (run with: pytest)

Test	What it checks
test_features: causality	Changing a future transaction never changes earlier features (no leakage)
test_features: velocity windows	1 h / 24 h / 7 d counts and ratios computed correctly
test_features: sandbox	What-if scoring and scenarios never change live card state
test_features: online/offline parity	API features equal training features on 3,000 replayed transactions
test_metrics	Confusion counts, F1/precision/cost thresholds, accuracy paradox
test_api	Health, tokenisation, what-if purity, suspicious > normal, stream, review, scenarios, validation errors

11 How to run, test, present and deploy

11.1 Run locally (Windows PowerShell)

```
git clone https://github.com/Prateek22672/AI-Based-Credit-Card-Fraud-Detection-System.git
cd AI-Based-Credit-Card-Fraud-Detection-System
python -m venv .venv
.venv\Scripts\pip install -r requirements.txt
# trained models + held-out test data are included in artifacts/, so the API runs immediately:
.venv\Scripts\python -m uvicorn fraudlens.api.main:app --port 8000
# second terminal: the dashboard
cd frontend
npm install
npm run dev          # open http://localhost:5173  (API docs: http://localhost:8000/docs)
```

11.2 Reproduce the research (optional, about 1.5 h on a laptop CPU)

```
.venv\Scripts\python -m fraudlens.download          # both datasets (~260 MB)
.venv\Scripts\pip install -r requirements-dl.txt --index-url https://download.pytorch.org/whl/cpu
.venv\Scripts\python -m fraudlens.pipeline all       # features, E1-E7, artifacts, scenarios
.venv\Scripts\python -m fraudlens.plots              # figures
.venv\Scripts\python -m reporting.build             # these Word + PDF reports
.venv\Scripts\python -m pytest                      # automated tests
```

11.3 Suggested demonstration (10 minutes)

1. **Overview page**: state the problem, the verdict (web platform + hybrid ML) and the headline numbers.
2. **Live Monitor**: press Start. Explain that these are unseen April-June 2020 transactions replayed in time order; watch alerts appear and live precision/recall converge towards the test results.
3. **Click a red (HIGH) row**: read the reasons aloud (for example "5 transactions in the past hour", "amount 9x the card's average") and point at the three detector votes.
4. **Scam Scenarios**: run Card testing, then Normal day. Show the risk climbing step by step for the attack and staying green for the normal customer.
5. **Transaction Tester**: take a real card, enter its usual amount (LOW), then change to $1,150 online at 02:30 (HIGH). This shows the model judges behaviour relative to the customer.
6. **Case Queue**: confirm one alert and dismiss one; open the audit record to show the forensic trail.
7. **Results page**: show E3 (how shortcuts inflate numbers) and E4 (features matter most), and explain how each claim is proven.
8. **Project Report page**: milestones and work distribution for the reviewers.

11.4 Deploy on your own domain

- **Simplest (one server / VPS):** `docker build -t fraudlens .` then `docker run -p 8000:8000 fraudlens`. The container serves the API and the built dashboard; point the domain (or a Cloudflare tunnel / Nginx reverse proxy) at port 8000.
- **Split hosting:** build the dashboard with `VITE_API_URL=https://api.your-domain` and upload `frontend/dist` to any static host (cPanel, Netlify, Vercel); run the API on a VPS or a Python host and set

`FRAUDLENS_CORS_ORIGINS=https://your-domain`.

- **Security for public demos:** set `FRAUDLENS_API_KEY` (and the same `VITE_API_KEY` at build time) and a private `FRAUDLENS_TOKEN_SECRET`.
- **Results without a server:** the Results and Project Report pages also work from static files, so reviewers can browse them even if the API is offline.

12 Limitations and threats to validity

- **Synthetic primary data.** Sparkov is simulated; its fraud patterns are cleaner than real ones, so absolute scores are higher than a bank would see. ULB is real but anonymised, preventing behavioural features.
- **Label delay is simplified** to fixed 7-day gaps; in reality labels arrive at varying times and investigator feedback is biased towards alerted transactions (Dal Pozzolo et al., 2018).
- **Geography is a weak signal in Sparkov** (merchants are placed near home at random), so impossible-travel detection relies on policy rule P2 and is not validated on real location data.
- **The safety-layer thresholds are set, not learned.** The rule parameters come from domain reasoning; the 99.95th-percentile novelty threshold was chosen after looking at its alert cost on the validation and test periods. These are business settings to review with a fraud team, not tuned detection results.
- **Card testing and cloned cards are demonstrated on replayed scenarios**, because Sparkov contains no labelled examples of them; their detection rate on real traffic is not measured.
- **Adversarial adaptation.** Fraudsters who learn the rules can mimic normal behaviour; periodic retraining and the anomaly layer mitigate but do not remove this.
- **Cost assumptions** ($5 per alert) are illustrative; the sensitivity table shows how decisions change with the assumption.
- **Social-engineering scams** where the real customer authorises payment are mostly out of reach of transaction features alone.

13 Future scope

Table 35. Future work

Extension	Value	Reference
Streaming ingestion (Kafka) + Redis feature store	Bank-scale throughput	Carcillo 2018 (SCARFF), T20
Compiled tree runtime (treelite / ONNX) for the two forests	Single-call latency from ~0.2 s to a few ms	Engineering
Continuous retraining from reviewed cases, drift alarms on PSI	Adapts to new fraud patterns	T11, Lebichot 2021, Gama 2014
Graph model over card-merchant-device links	Detects organised rings and merchant compromise	T14, Liu 2021 (PC-GNN)
LSTM/Transformer over each card's transaction sequence	Richer behavioural memory	T19, Jurgovsky 2018
Mobile customer app ("Was this you?" push alerts)	Faster confirmation, fewer false blocks	Same REST API
Device, IP and payee signals	Social-engineering and account-takeover coverage	Industry practice
Validation on IEEE-CIS and real bank data	External validity	T2, T6, T9

14 Conclusion

FraudLens shows that an explainable hybrid of gradient-boosted trees, a random forest and an anomaly detector, fed with causal behavioural features, detects card fraud accurately (test PR-AUC 0.979, recall 92.5%, precision 94.6%), fast (209.4 ms per transaction) and transparently, and that it is deployable as a real web service. Just as important, the project shows *how to evaluate* fraud detectors honestly: several impressive published numbers can only be reproduced with data leakage, while our results hold on unseen future data, with confidence intervals and significance tests. Existing projects mainly detect fraud; FraudLens detects it, recognises unusual behaviour, prices the risk, explains the decision and leaves an audit trail.

References

1. Andrea Dal Pozzolo, Olivier Caelen, Reid A. Johnson, Gianluca Bontempi (2015). Calibrating Probability with Undersampling for Unbalanced Classification. *2015 IEEE Symposium Series on Computational Intelligence (SSCI), Cape Town*, pp. 159-166. https://doi.org/10.1109/SSCI.2015.33
2. Andrea Dal Pozzolo, Giacomo Boracchi, Olivier Caelen, Cesare Alippi, Gianluca Bontempi (2018). Credit Card Fraud Detection: A Realistic Modeling and a Novel Learning Strategy. *IEEE Transactions on Neural Networks and Learning Systems*, vol. 29, no. 8, pp. 3784-3797. https://doi.org/10.1109/TNNLS.2017.2736643
3. Yann-Ael Le Borgne, Wissam Siblini, Bertrand Lebichot, Gianluca Bontempi (2022). Reproducible Machine Learning for Credit Card Fraud Detection - Practical Handbook. *Universite Libre de Bruxelles (online book, Jupyter Book)*
4. Christopher Whitrow, David J. Hand, Piotr Juszczak, David Weston, Niall M. Adams (2009). Transaction aggregation as a strategy for credit card fraud detection. *Data Mining and Knowledge Discovery*, vol. 18, no. 1, pp. 30-55. https://doi.org/10.1007/s10618-008-0116-z
5. Alejandro Correa Bahnsen, Djamila Aouada, Aleksandar Stojanovic, Bjorn Ottersten (2016). Feature engineering strategies for credit card fraud detection. *Expert Systems with Applications*, vol. 51, pp. 134-142. https://doi.org/10.1016/j.eswa.2015.12.030
6. Fabrizio Carcillo, Yann-Ael Le Borgne, Olivier Caelen, Yacine Kessaci, Frederic Oble, Gianluca Bontempi (2021). Combining unsupervised and supervised learning in credit card fraud detection. *Information Sciences*, vol. 557, pp. 317-331 (online 2019). https://doi.org/10.1016/j.ins.2019.05.042
7. Fei Tony Liu, Kai Ming Ting, Zhi-Hua Zhou (2008). Isolation Forest. *2008 Eighth IEEE International Conference on Data Mining (ICDM), Pisa*, pp. 413-422. https://doi.org/10.1109/ICDM.2008.17
8. Leo Breiman (2001). Random Forests. *Machine Learning*, vol. 45, no. 1, pp. 5-32. https://doi.org/10.1023/A:1010933404324
9. Tianqi Chen, Carlos Guestrin (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*, pp. 785-794. https://doi.org/10.1145/2939672.2939785
10. Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, Tie-Yan Liu (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems 30 (NIPS 2017)*
11. David H. Wolpert (1992). Stacked generalization. *Neural Networks*, vol. 5, no. 2, pp. 241-259. https://doi.org/10.1016/S0893-6080(05)80023-1
12. Scott M. Lundberg, Su-In Lee (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems 30 (NIPS 2017)*
13. Scott M. Lundberg, Gabriel Erion, Hugh Chen, Alex DeGrave, Jordan M. Prutkin, Bala Nair, Ronit Katz, Jonathan Himmelfarb, Nisha Bansal, Su-In Lee (2020). From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56-67. https://doi.org/10.1038/s42256-019-0138-9
14. Ismini Psychoula, Andreas Gutmann, Pradip Mainali, Sharon H. Lee, Paul Dunphy, Fabien Petitcolas (2021). Explainable Machine Learning for Fraud Detection. *Computer (IEEE)*, vol. 54, no. 10, pp. 49-59. https://doi.org/10.1109/MC.2021.3081249
15. Leo Grinsztajn, Edouard Oyallon, Gael Varoquaux (2022). Why do tree-based models still outperform deep learning on typical tabular data?. *Advances in Neural Information Processing Systems 35 (NeurIPS 2022), Datasets and Benchmarks Track*, pp. 507-520 (Curran). https://doi.org/10.52202/068431-0037
16. Ravid Shwartz-Ziv, Amitai Armon (2022). Tabular data: Deep learning is not all you need. *Information Fusion*, vol. 81, pp. 84-90. https://doi.org/10.1016/j.inffus.2021.11.011

17. Takaya Saito, Marc Rehmsmeier (2015). The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLoS ONE*, vol. 10, no. 3, e0118432. https://doi.org/10.1371/journal.pone.0118432
18. Davide Chicco, Giuseppe Jurman (2020). The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. *BMC Genomics*, vol. 21, article 6. https://doi.org/10.1186/s12864-019-6413-7
19. Miriam Seoane Santos, Jastin Pompeu Soares, Pedro Henriques Abreu, Helder Araujo, Joao Santos (2018). Cross-Validation for Imbalanced Datasets: Avoiding Overoptimistic and Overfitting Approaches. *IEEE Computational Intelligence Magazine*, vol. 13, no. 4, pp. 59-76. https://doi.org/10.1109/MCI.2018.2866730
20. Shachar Kaufman, Saharon Rosset, Claudia Perlich, Ori Stitelman (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, article 15, pp. 1-21. https://doi.org/10.1145/2382577.2382579
21. Khizar Hayat, Baptiste Magnier (2025). Data Leakage and Deceptive Performance: A Critical Examination of Credit Card Fraud Detection Methodologies. *Mathematics (MDPI)*, vol. 13, no. 16, article 2563. https://doi.org/10.3390/math13162563
22. Ruben van den Goorbergh, Maarten van Smeden, Dirk Timmerman, Ben Van Calster (2022). The harm of class imbalance corrections for risk prediction models: illustration and simulation using logistic regression. *Journal of the American Medical Informatics Association*, vol. 29, no. 9, pp. 1525-1534. https://doi.org/10.1093/jamia/ocac093
23. Charles Elkan (2001). The Foundations of Cost-Sensitive Learning. *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI-01), Seattle*, pp. 973-978
24. Alexandru Niculescu-Mizil, Rich Caruana (2005). Predicting good probabilities with supervised learning. *Proceedings of the 22nd International Conference on Machine Learning (ICML '05), Bonn*, pp. 625-632. https://doi.org/10.1145/1102351.1102430
25. Sebastiaan Hoppner, Bart Baesens, Wouter Verbeke, Tim Verdonck (2022). Instance-dependent cost-sensitive learning for detecting transfer fraud. *European Journal of Operational Research*, vol. 297, no. 1, pp. 291-300. https://doi.org/10.1016/j.ejor.2021.05.028
26. Toon Vanderschueren, Tim Verdonck, Bart Baesens, Wouter Verbeke (2022). Predict-then-optimize or predict-and-optimize? An empirical evaluation of cost-sensitive learning strategies. *Information Sciences*, vol. 594, pp. 400-415. https://doi.org/10.1016/j.ins.2022.02.021
27. Joao Gama, Indre Zliobaite, Albert Bifet, Mykola Pechenizkiy, Abdelhamid Bouchachia (2014). A survey on concept drift adaptation. *ACM Computing Surveys*, vol. 46, no. 4, article 44, pp. 1-37. https://doi.org/10.1145/2523813
28. Johannes Jurgovsky, Michael Granitzer, Konstantin Ziegler, Sylvie Calabretto, Pierre-Edouard Portier, Liyun He-Guelton, Olivier Caelen (2018). Sequence classification for credit-card fraud detection. *Expert Systems with Applications*, vol. 100, pp. 234-245. https://doi.org/10.1016/j.eswa.2018.01.037
29. Fabrizio Carcillo, Andrea Dal Pozzolo, Yann-Ael Le Borgne, Olivier Caelen, Yannis Mazzer, Gianluca Bontempi (2018). SCARFF: a scalable framework for streaming credit card fraud detection with Spark. *Information Fusion*, vol. 41, pp. 182-194. https://doi.org/10.1016/j.inffus.2017.09.005
30. Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, W. Philip Kegelmeyer (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, vol. 16, pp. 321-357. https://doi.org/10.1613/jair.953
31. Mohammad Shanaa, Sherief Abdallah (2025). A Hybrid Anomaly Detection Framework Combining Supervised and Unsupervised Learning for Credit Card Fraud Detection. *F1000Research*, vol. 14, article 664 (v2, Dec 2025). https://doi.org/10.12688/f1000research.166350.2
32. Yang Liu, Xiang Ao, Zidi Qin, Jianfeng Chi, Jinghua Feng, Hao Yang, Qing He (2021). Pick and Choose: A GNN-based Imbalanced Learning Approach for Fraud Detection. *Proceedings of the Web Conference 2021 (WWW*

'21), pp. 3168-3177. https://doi.org/10.1145/3442381.3449989

33. The Nilson Report, press release 'Global Card Fraud Losses at $33 Billion' (GlobeNewswire) (2026-01-07). Global payment-card fraud losses (issuers, merchants, acquirers, ATM processors) in 2024. https://www.globenewswire.com/news-release/2026/01/07/3214821/0/en/global-card-fraud-losses-at-33-billion.html
34. Le Borgne et al., Reproducible Machine Learning for Credit Card Fraud Detection - Practical Handbook, Foreword (citing Nilson Report 2019) (2022). Long-run growth of global card fraud losses (historical trend). https://fraud-detection-handbook.github.io/fraud-detection-handbook/Foreword.html
35. EBA-ECB joint 2025 Report on Payment Fraud, ECB press release (2025-12-15). Payment fraud in the European Economic Area, 2024. https://www.ecb.europa.eu/press/pr/date/2025/html/ecb.pr251215~e133d9d683.en.html
36. European Central Bank, Seventh report on card fraud (2021-10). Card-not-present (CNP) fraud share of total card fraud value (Europe). https://www.ecb.europa.eu/press/cardfraud/html/ecb.cardfraudreport202110~cac4c418e8.en.html
37. UK Finance, Annual Fraud Report 2025 press release (2025-05-28). UK unauthorised fraud and remote-purchase (CNP) fraud, 2024. https://www.ukfinance.org.uk/news-and-insight/press-release/fraud-report-2025-press-release
38. Reserve Bank of India, Annual Report 2025-26, Chapter VI (Regulation, Supervision and Financial Stability), Tables VI.2 and VI.3 (2026-05-29). India: bank frauds reported to RBI (frauds of Rs 1 lakh and above), latest three years. https://www.rbi.org.in/Scripts/AnnualReportPublications.aspx?Id=1466
39. RBI Annual Report 2024-25 as reported by Deccan Herald and SBI Ecowrap (secondary sources; the RBI PDF is behind a bot-check) (2025-05-29). India: bank fraud figures as first published in the previous RBI Annual Report (before revisions). https://www.deccanherald.com/business/bank-frauds-value-trebles-to-over-rs-36000-crore-3563349
40. Reserve Bank of India notification (2017-07-06). India: customer liability rules for unauthorised electronic banking transactions. https://www.rbi.org.in/commonman/english/scripts/Notification.aspx?Id=2336
41. T1. Shobhan Banoth, K. Reddy Madhavi (2024). A Novel Deep Learning Framework for Credit Card Fraud Detection. *2024 13th International Conference on System Modeling & Advancement in Research Trends (SMART), Moradabad, India, 6-7 Dec 2024, pp. 191-196 (IEEE)*. https://doi.org/10.1109/SMART63812.2024.10882509
42. T2. Pranav Prakash Menon, Amit Sachdeva, Gayathri V. M (2024). Fraud Shield: Credit Card Fraud Detection with Ensemble and Deep Learning. *2024 4th International Conference on Pervasive Computing and Social Networking (ICPCSN), Salem, India, 3-4 May 2024, pp. 224-230 (IEEE)*. https://doi.org/10.1109/ICPCSN62568.2024.00045
43. T3. Bipin Bihari Jayasingh, G. Bhagya Sri (2023). Online Transaction Anomaly Detection Model for Credit Card Usage Using Machine Learning Classifiers. *2023 International Conference on Emerging Smart Computing and Informatics (ESCI), Pune, India, pp. 1-5 (IEEE)*. https://doi.org/10.1109/ESCI56872.2023.10100152
44. T4. Rohan Pawar, Hardik Kathuria, Praveen Joe I R (2023). Credit Card Fraud Detection and Analysis. *2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT), Delhi, India, 6-8 Jul 2023, pp. 1-5 (IEEE)*. https://doi.org/10.1109/ICCCNT56998.2023.10306410
45. T5. Suni Jose, Deepa Devassy, Anly M Antony (2023). Detection of Credit Card Fraud Using Resampling and Boosting Technique. *2023 Advanced Computing and Communication Technologies for High Performance Applications (ACCTHPA), Ernakulam, India, pp. 1-8 (IEEE)*. https://doi.org/10.1109/ACCTHPA57160.2023.10083376
46. T6. Sreenivas Prasad B, Akash Babu N, Harthikeswar Reddy, Rimjhim Padam Singh, Sneha Kanchan (2024). A Machine Learning Approach for Credit Card Fraud Detection in Massive Datasets Using SMOTE and Random Sampling. *2024 IEEE Recent Advances in Intelligent Computational Systems (RAICS), Kothamangalam, Kerala, India, 16-18 May 2024, pp. 1-8*. https://doi.org/10.1109/RAICS61201.2024.10690025
47. T7. Prachi Gupta, Shatakshi Shukla, Vaishali Kikan, Ashwni Kumar (2024). Enhancing Fraud Detection in Credit Card Transactions using XGBoost and SMOTE: A Comparative Study. *2024 IEEE International Conference on*

Smart Power Control and Renewable Energy (ICSPCRE), Rourkela, India, 19-21 Jul 2024, pp. 1-6. https://doi.org/10.1109/ICSPCRE62303.2024.10675080

48. T8. Qin Wang, Mary Jane C. Samonte, Weiqing Diao, Yongjian Zhang (2024). Design and Empirical Analysis of Data-Driven Machine Learning Models for Credit Card Fraud Detection. *2024 5th International Conference on Big Data, Artificial Intelligence and Internet of Things Engineering (ICBAIE), Shenzhen, China, 25 Oct 2024, pp. 114-117 (IEEE)*. https://doi.org/10.1109/ICBAIE63306.2024.11116991
49. T9. Xiong Kewei, Binhui Peng, Yang Jiang, Tiying Lu (2021). A Hybrid Deep Learning Model For Online Fraud Detection. *2021 IEEE International Conference on Consumer Electronics and Computer Engineering (ICCECE), Guangzhou, China, pp. 431-434*. https://doi.org/10.1109/ICCECE51280.2021.9342110
50. T10. Ziyan Zhang, Shuyan Huang (2020). Credit Card Fraud Detection via Deep Learning Method Using Data Balance Tools. *2020 International Conference on Computer Science and Management Technology (ICCSMT), Shanghai, China, pp. 133-137 (IEEE)*. https://doi.org/10.1109/ICCSMT51754.2020.00033
51. T11. Fabrizio Carcillo, Yann-Ael Le Borgne, Olivier Caelen, Gianluca Bontempi (2017). An Assessment of Streaming Active Learning Strategies for Real-Life Credit Card Fraud Detection. *2017 IEEE International Conference on Data Science and Advanced Analytics (DSAA), Tokyo, Japan, pp. 631-639*. https://doi.org/10.1109/DSAA.2017.10
52. T12. Yinze Yang, Yibing Yu, Tingxi Li (2022). Deep Learning Techniques for Financial Fraud Detection. *2022 14th International Conference on Computer Research and Development (ICCRD), Shenzhen, China, 7-9 Jan 2022, pp. 16-22 (IEEE)*. https://doi.org/10.1109/ICCRD54409.2022.9730314
53. T13. Mehdi Hosseini Chagahi, Abolfazl Yekaneh, Saeed Mohammadi Dashtaki, Behzad Moshiri, Md. Jalil Piran (2026). XAI for Fraud Detection: An Attention-Based Ensemble of CNNs, GNNs, and Confidence-Driven Gating for Reliable Decision-Making. *IEEE Transactions on Information Forensics and Security, vol. 21, pp. 347-356*. https://doi.org/10.1109/TIFS.2025.3646480
54. T14. Munikrishnaiah Sundararamaiah, Chandrasekhar Rao Katru, Chandra Sekhar Kolli, Kruti Sutaria, K. B. Venkata Brahma Rao, Ramya Maranan (2025). Similarity-Navigated Graph Neural Network-Based Fraud Detection in Credit Card Transactions Optimized with the Greylag Goose Algorithm. *2025 International Conference on Machine Learning and Autonomous Systems (ICMLAS), Prawet, Thailand, 10-12 Mar 2025, pp. 1319-1325 (IEEE)*. https://doi.org/10.1109/ICMLAS64557.2025.10968459
55. T15. Mayank Pathak, Rohit Beniwal (2025). A Systematic Literature Survey on Credit Card Fraud Detection using Machine Learning Techniques. *2025 5th International Conference on Pervasive Computing and Social Networking (ICPCSN), Salem, India, 14-16 May 2025, pp. 149-154 (IEEE)*. https://doi.org/10.1109/ICPCSN65854.2025.11035471
56. T16. Shubham Saini, Gaurav Bathla, Vikas Wasson (2024). Credit Card Fraud Detection using Machine Learning: Methods, Challenges and Solutions. *2024 International BIT Conference (BITCON), Dhanbad, India, 7 Dec 2024, pp. 1-6 (IEEE)*. https://doi.org/10.1109/BITCON63716.2024.10985587
57. T17. Shruti Chawla, Sidhant Singh Bhadauriya, Abhishake Bansal, Chetna Sharma, Nomita Patwal (2024). Advancement in Credit Card Fraud Detection: A Comprehensive Review of Machine Learning Techniques. *2024 1st International Conference on Sustainable Computing and Integrated Communication in Changing Landscape of AI (ICSCAI), Greater Noida, India, 4-6 Jul 2024, pp. 1-7 (IEEE)*. https://doi.org/10.1109/ICSCAI61790.2024.10866665
58. T18. Vishnu R. Sonwane, Siddika Zanje, Siddhant Yenpure, Yash Gunjal, Yash Kulkarni, Rohit Yeole (2024). Advanced Machine Learning Techniques for Credit Card Fraud Detection: A Comprehensive Study. *2024 5th International Conference on Smart Electronics and Communication (ICOSEC), Trichy, India, 18-20 Sep 2024, pp. 1978-1981 (IEEE)*. https://doi.org/10.1109/ICOSEC61587.2024.10722667
59. T19. Ibomoiye Domor Mienye, Nobert Jere (2024). Deep Learning for Credit Card Fraud Detection: A Review of Algorithms, Challenges, and Solutions. *IEEE Access, vol. 12, pp. 96893-96910*. https://doi.org/10.1109/ACCESS.2024.3426955

60. T20. Asma Cherif, Arwa Badhib, Heyfa Ammar, Suhair Alshehri, Manal Kalkatawi, Abdessamad Imine (2023). Credit card fraud detection in the era of disruptive technologies: A systematic review. *Journal of King Saud University - Computer and Information Sciences, vol. 35, no. 1, pp. 145-174 (Elsevier / King Saud University)*. https://doi.org/10.1016/j.jksuci.2022.11.008

Appendix A: Configuration and hyper-parameters

Table 36. Key settings (fraudlens/config.py, models.py, hybrid.py)

Setting	Value
Split	train <= 2019-12-31; validation 2020-01-08 to 2020-03-31; test >= 2020-04-08
XGBoost	hist, depth 6, learning rate 0.1, subsample 0.8, colsample 0.8, scale_pos_weight = neg/pos, early stopping 50 rounds on validation PR-AUC
Random Forest	200 trees, max depth 16, min leaf 5, sqrt features, 30% bootstrap samples, balanced_subsample weights
Isolation Forest	300 trees, 1,024 samples per tree, fitted on genuine training transactions, 18 behaviour features
LightGBM	63 leaves, learning rate 0.1, class weight neg/pos with max_delta_step 0.7 (without it, training is numerically unstable at a ~177x weight), early stopping 100 rounds
Safety layer	P1: 3+ txns in 1 h and amount >= 5x card average; P2: >= 500 km at > 800 km/h; P3: < $5 online after 3+ txns in 1 h; N1: anomaly percentile >= 99.95%
Meta-learner	Logistic regression (C = 1) on [logit p_xgb, logit p_rf, anomaly z-score], 5-fold out-of-fold for thresholds
Thresholds	alert = cost-optimal at $5/alert; block = lowest threshold with >= 90% precision
Deep baselines	MLP 256-128-64 with batch norm and dropout (up to 15 epochs); 1D-CNN 32/64 channels (up to 8 epochs, CPU budget); Adam 1e-3; batch 2048; positive-class weight; early stopping on validation PR-AUC
Statistics	500 bootstrap resamples for CIs and paired tests; seed 42

Appendix B: Work distribution

Table 37. Fill in before submission

Module	Owner	Contribution
Literature review and mapping	[Member 1]	[describe]
Data, EDA and feature engineering	[Member 2]	[describe]
Models, hybrid engine and experiments	[Member 3]	[describe]
API, dashboard and deployment	[Member 4]	[describe]
Documentation and presentation	[All]	[describe]